

Mobile Application Penetration Testing Techniques

Alper Basaran

20+ Years in Cybersecurity

Mostly Penetration Testing and Training

CISSP, CISA, GICSP, OSWP, CPTE, eCIR, etc...

Key Concepts

- + Penetration testing methodologies for mobile apps
- + Dynamic testing for Android and iOS
- + Real-world attack simulation and MASVS alignment
- + API and backend testing for mobile applications
- + Reporting and remediation strategies

MAJOR TOPICS

- + Mobile security flaws (M1–M10) and real-world attack vectors
- + Android & iOS dynamic testing environments
- + Testing tools: Drozer, ADB, Logcat, SQLite, and more
- + API pentesting and advanced runtime attacks
- + Simulating complete engagements and reporting



LEARNING OUTCOMES

- + Identify and exploit vulnerabilities in mobile apps using real tools
- + Map findings to MASVS and industry standards
- + Execute advanced dynamic testing on Android and iOS
- + Conduct and document complete mobile penetration tests

- + Familiarity with mobile operating systems
- + Basic cybersecurity and networking knowledge



LET'S GO!





Introduction & Methodology

Shifting from Traditional Security Thinking

- + Mobile-first security perspective
- + Always-on connectivity
- + Personal device implications
- + Trust boundaries
- + Attack surface expansion

Understanding the Mobile Pentesting Mindset

- + Balancing usability vs. security in mobile development
- + Real-world adversary motivations
- + Common mistakes developers make
- + Developing the "app + ecosystem" mindset

Not “Just a Smaller Web App” Web App

POST /login HTTP/1.1

Host: webapp.com

Content-Type: application/x-www-form-urlencoded

email=alice@example.com&password=123456

Not “Just a Smaller Web App” Web App

POST /login HTTP/1.1

Host: webapp.com

Content-Type: application/x-www-form-urlencoded

email=alice@example.com&password=123456

Set-Cookie: PHPSESSID=abc123xyz;

Not “Just a Smaller Web App” Mobile App

```
POST /api/auth/login HTTP/1.1  
Content-Type: application/json
```

```
{  
  "email": "alice@example.com",  
  "password": "123456"  
}
```

Not “Just a Smaller Web App” Mobile App

```
POST /api/auth/login HTTP/1.1  
Content-Type: application/json
```

```
{  
  "email": "alice@example.com",  
  "password": "123456"  
}
```

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",  
  "refresh_token": "d41d8cd98f00b204e9800998ecf8427e",  
  "expires_in": 3600  
}
```

Not “Just a Smaller Web App”

Mobile App: Security Features

- + Short-lived access tokens, long-lived refresh tokens
- + TLS enforced
- + No cookie-based auth → immune to CSRF
- + App attestation or fingerprinting of devices
- + MFA enforced on server side
- + Token revocation capabilities
- + Biometric unlocking of the app for silent auth

Not “Just a Smaller Web App”

Mobile App: Pentest Targets

- + Insecure storage of tokens (e.g., in SharedPreferences or plist files)
- + Token leakage via logs or crash reports
- + Reverse engineering to find API keys or token formats
- + JWT token tampering (e.g., none algorithm, weak secret)
- + Improper refresh token flow
- + Session re-use across devices
- + Exposed hardcoded secrets used in auth process

Mobile vs. Web & Traditional App Pentesting

Web App Penetration Testing	Mobile App Penetration Testing
Server-focused	Client-focused
Lives in the browser	OS-managed permissions
Network attacks	Local attacks
Static web code	Binaries (.ipa/.apk)
Easy to proxy	May be difficult/impossible to proxy

Unique Mobile Attack Vectors

- + Sensors and hardware access
- + Inter-app communication
- + Physical security considerations
- + Offline operation
- + Platform-specific vulnerabilities

API Interaction Differences

- + Mobile backend patterns
- + Authentication token management
- + API defense patterns
- + Traffic analysis challenges
- + Mobile-specific API endpoints

Common Mobile Threat Actors

- + Malicious applications
- + Network attackers
- + Physical access attackers
- + Cloud service attackers
- + Insider threats
- + Advanced persistent threats

Legal Considerations and Responsible Disclosure

- + **Computer Fraud and Abuse Act (CFAA)** implications: Understanding legal boundaries
- + **Digital Millennium Copyright Act (DMCA)** considerations: Reverse engineering legality
- + **End User License Agreement (EULA)** restrictions: Legal constraints in app testing
- + **International testing considerations**: Varying legal frameworks across jurisdictions
- + **Telecommunications regulations**: Legal aspects of testing carrier-related functionality
- + **Data protection laws**: GDPR, CCPA, and other privacy law implications





Lab Setup for Dynamic Testing

Real Devices vs. Emulators/Simulators

Feature	Real Device	Emulator
Performance	Real	Can be inaccurate
Root/Jailbreak	Possible?	Easier
Hardware Testing	Full	Limited
App Store Integration	Full	May be limited
Cert Pinning Bypass	Hard but realistic	Easier

Setup

- + Android:
 - + Android Studio + CLI tools
- + iOS
 - + ...



Setup

- + Android:
 - + Android Studio + CLI tools
- + iOS
 - + MAC + Older iPhone + CLI tools

Setup

+ In conclusion: It depends

A person wearing glasses is shown from a high angle, looking down at a laptop. The scene is illuminated with blue and purple light, creating a tech-oriented atmosphere. The person's hands are visible on the laptop keyboard.

OWASP Mobile Top 10 Vulnerabilities

OWASP Mobile Top 10

M1: Improper Credential Usage

Learning Objectives

By the end of this topic, you will be able to:

- + Identify insecure storage and transmission of credentials in mobile apps.
- + Understand the risks of hardcoded credentials and insecure authentication flows.
- + Perform credential harvesting through static and dynamic analysis.
- + Implement secure authentication practices in mobile development.

What is “Improper Credential Usage”?

- + Insecure practices in the collection, storage, handling, and transmission of authentication credentials
- + This vulnerability enables attackers to:
 - + Steal credentials and impersonate users
 - + Hijack sessions or access backend APIs
 - + Escalate privileges
 - + Maintain long-term persistence on backend services

Types of Credentials Commonly Affected

Credential Type	Description	Typical Misuse
Username/Passwords	Primary authentication values	Stored in plaintext, hardcoded, or sent over HTTP
Session Tokens	JWTs, OAuth tokens, cookies	Logged, cached, or never refreshed
API Keys/Secrets	For calling third-party services	Hardcoded in app or reverse-engineered
Biometric Data	Fingerprint/FaceID, fallback PINs	Local-only checks, easy to bypass
Device Identifiers	IMEI, UDID	Used as insecure auth token

Remember the Android Security Architecture

- + **Application sandboxing:** Each app runs as a unique UID; can't access other apps' data by default.
- + **Internal storage protection:** Data stored under /data/data/<package> is private unless rooted.
- + **Keystore system:** Used for storing cryptographic keys bound to hardware-backed security.
- + **SELinux enforcement:** Prevents unauthorized access to system resources.

Remember the iOS Security Architecture

- + **App sandboxing and code signing:** Enforces strong process and filesystem isolation.
- + **Keychain Services:** Secure credential storage with encryption tied to device passcode or FaceID.
- + **Secure Enclave:** Dedicated coprocessor for secure operations and biometric storage.
- + **Data Protection Classes:** Files encrypted with different access controls (e.g., when unlocked).

How Improper Credential Usage Happens

- + Hardcoded Credentials
 - + API keys, tokens, or user credentials are embedded directly in the codebase.
- + Credentials stored in:
 - + SharedPreferences, NSUserDefaults
 - + SQLite or Realm databases
 - + Unencrypted files

How Improper Credential Usage Happens

- + Sending credentials over:
 - + Plain HTTP
 - + Improper TLS configurations (e.g., no hostname verification)
- + Leaked in Logs / Intents / Notifications
 - + Developers may log credentials during debugging.
 - + Intents can leak sensitive data if exported=true without permission controls.

How Improper Credential Usage Happens

- + Improper Biometric / PIN Fallback
 - + Some apps allow local-only verification (e.g., `isFingerprintValid()` returns true).
 - + Lacking server-side revalidation after unlock.
- + Long-Lived Tokens & Poor Session Management
 - + Tokens never expire or are reused indefinitely.
 - + Not tied to device/user/IP or revocable.

Common Exploitation Techniques

(1/2)

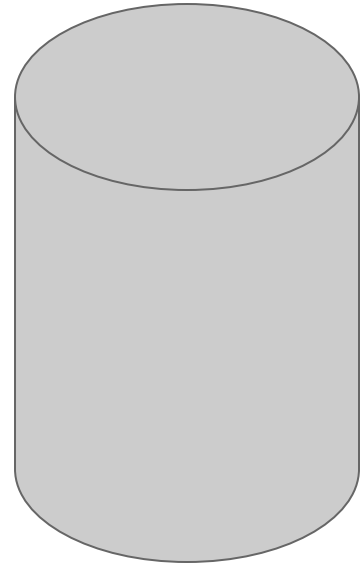
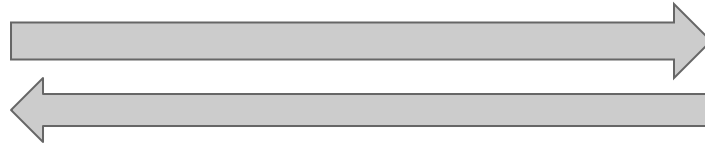
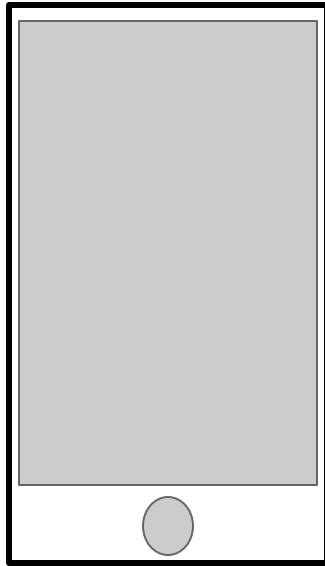
- + APK Decompilation: Analyze Java/Kotlin source for hardcoded secrets
- + Frida Runtime Hooking: Bypass biometric auth or intercept credential flows
- + Logcat Monitoring : Capture tokens/passwords logged during login
- + Memory Dumping : Dump app memory to search for tokens/creds
- + Intercepting Traffic: Use Burp with SSL pinning bypass to catch cleartext or weakly protected credentials

Common Exploitation Techniques

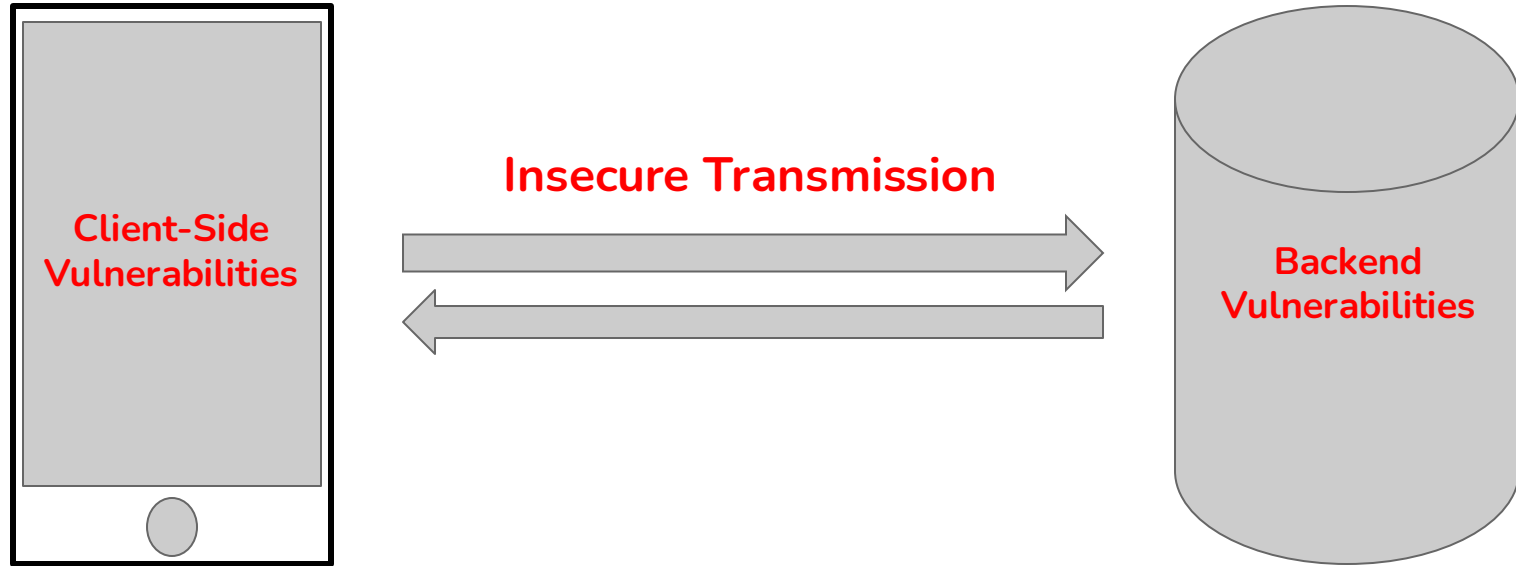
(2/2)

- + Intercepting Traffic: Use Burp with SSL pinning bypass to catch cleartext or weakly protected credentials
- + Local Storage Analysis: Search device storage for unencrypted data
- + Intent Hijacking: Catch sensitive data passed between components

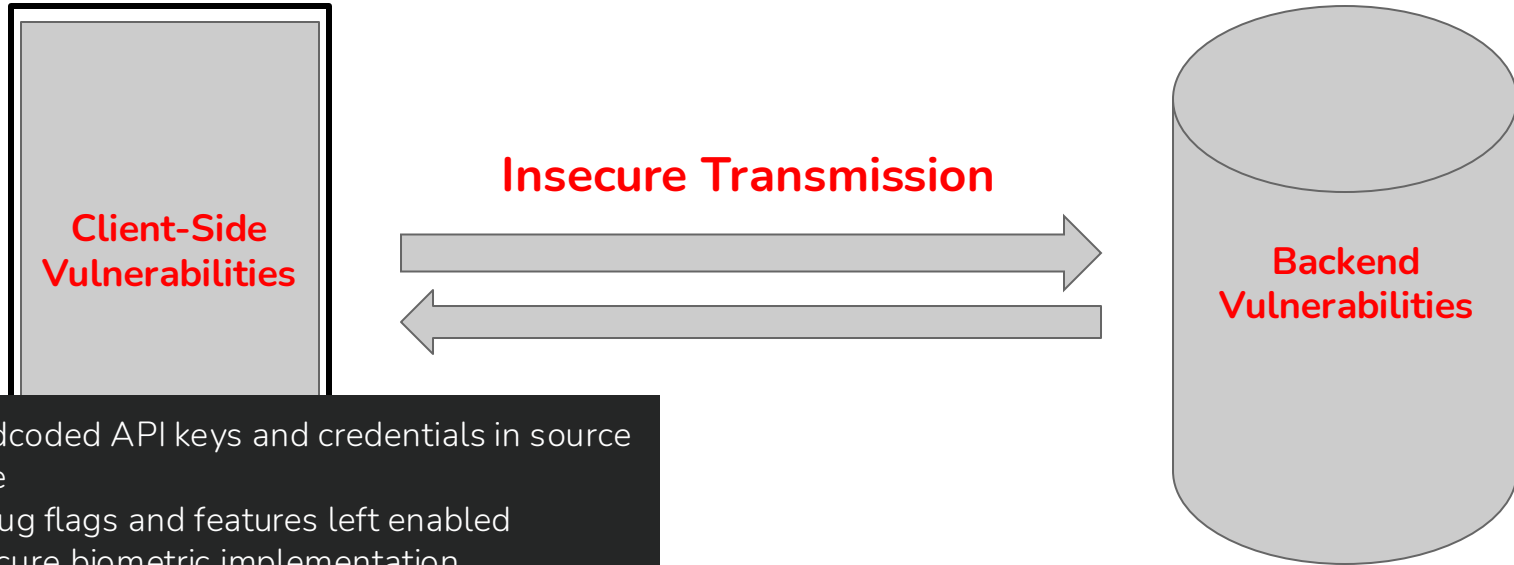
Attack Vectors



Attack Vectors

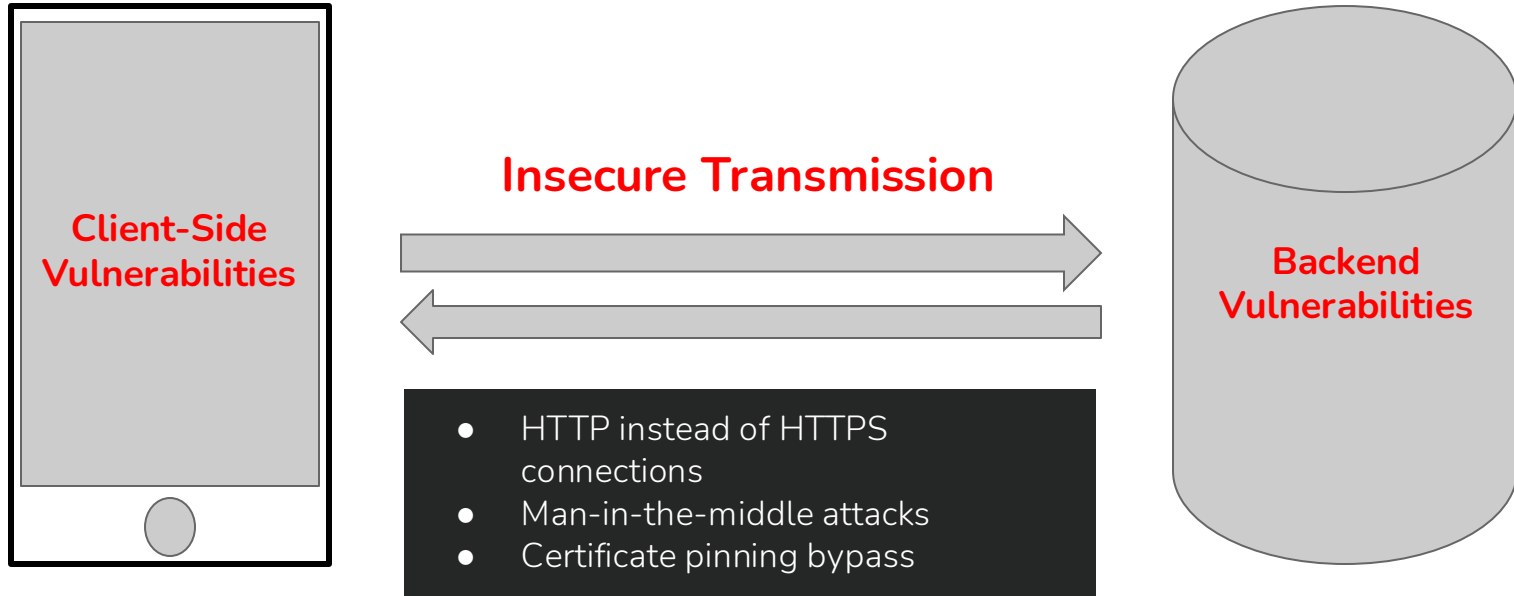


Attack Vectors

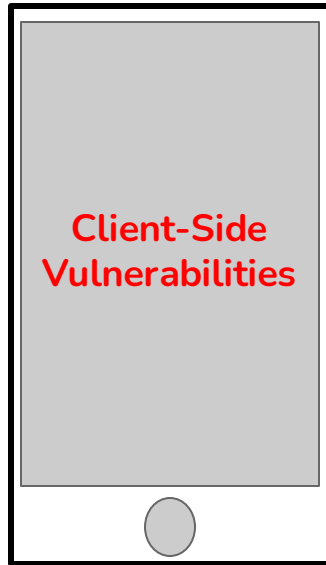


- Hardcoded API keys and credentials in source code
- Debug flags and features left enabled
- Insecure biometric implementation
- Clipboard vulnerabilities

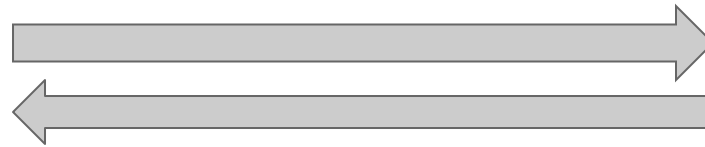
Attack Vectors



Attack Vectors



Insecure Transmission



- Weak password policies
- Lack of brute force protection
- Insecure session management
- Vulnerable password reset mechanisms



M1: Improper Credential Usage

Key Takeaways

- + Credentials are the crown jewels.
- + The mobile OS offers tools for secure storage and access control
- + Dynamic analysis reveals the truth about how credentials are actually handled in real runtime scenarios.
- + Reverse engineering is often faster than brute force attacks.
- + Improper credential usage is almost always due to developer mistakes

OWASP Mobile Top 10
M2: Inadequate Supply Chain Security

Assessment Methodology

- + Encompasses all elements involved in developing, building, distributing, and maintaining mobile applications.
 - + SDKs and libraries (e.g., analytics, ads, authentication)
 - + APIs from third-party vendors
 - + Build tools and package managers (e.g., Gradle, CocoaPods, npm)
 - + Cloud services and CI/CD pipelines

The Growing Importance of Supply Chain Security

- + Mobile apps typically incorporate 20-30 third-party components
- + Average mobile app connects to 15+ distinct backend services
- + Over 80% of code in typical mobile apps comes from third-party sources
- + Increasing trend of supply chain attacks as direct penetration becomes harder

Common Characteristics

- + Use of outdated or vulnerable third-party SDKs
- + SDKs that silently exfiltrate data
- + Third-party code executing without sandboxing
- + Unverified binaries included in mobile apps
- + No Software Bill of Materials (SBOM)

MITRE ATT&CK Mobile Mapping (Relevant TTPs)

ID	Tactic	Technique
T1476	Initial Access	Supply Chain Compromise
T1444	Collection	Credential Storage via 3rd-Party Keychain
T1456	Collection	Input Capture through 3rd-Party Keyboard
T1437	Defense Evasion	App Behavior Modification
T1430	Credential Access	Side-loaded/Third-Party App Abuse

Vulnerability Classes Under M2

- + Malicious SDKs: SDKs that perform unauthorized data collection or dynamic code loading
- + Vulnerable Dependencies: Outdated libraries with known CVEs
- + Tampered Libraries: Libraries replaced in transit or compromised in repo
- + Insecure CI/CD Pipelines: Weak code signing, missing artifact validation
- + Unverified External Code: No hash/pin check for fetched JS or WebView content

Security Implications in Dynamic Analysis

- + In dynamic testing, supply chain issues may surface as:
 - + Unexpected domains being contacted (via Burp or MITM proxy)
 - + Permissions requested by SDKs unrelated to app logic (e.g., a weather SDK requesting contact access)
 - + Obfuscated third-party code that hides runtime behavior
 - + Encrypted payloads to 3rd-party servers not under developer control

Indicators of Risky Supply Chain Integration

- + No version pinning: Automatically pulls latest version, including compromised ones
- + SDK obfuscation: Impossible to audit behavior
- + External JS fetched by WebView: Prone to remote injection
- + Library with access to ACCESS_FINE_LOCATION, READ_CONTACTS
- + SDK without privacy policy or open-source code: No visibility into behavior
- + No SBOM or list of dependencies: No accountability chain for issues



Attack Vectors

(0/4)

- + Third-Party Library Vulnerabilities
- + Build Process Compromises
- + Distribution Platform Issues
- + Update Mechanism Flaws

Attack Vectors

(1/4)

- + **Third-Party Library Vulnerabilities**
 - + Outdated libraries with known security flaws
 - + Malicious code insertion in popular libraries
 - + Typosquatting and dependency confusion attacks
 - + Unmaintained libraries with no security patches

Attack Vectors

(2/4)

- + **Build Process Compromises**
 - + Compromised build servers and CI/CD pipelines
 - + Tampered build tools or compiler backdoors
 - + Insecure build configurations
 - + Modified manifest files

Attack Vectors

(3/4)

- + **Distribution Platform Issues**
 - + App store security bypasses
 - + Fake or repackaged apps
 - + Insecure app distribution channels
 - + App store account compromises

Attack Vectors

(4/4)

- + **Update Mechanism Flaws**
 - + Insecure update verification
 - + Man-in-the-middle vulnerabilities during updates
 - + Downgrade attacks
 - + Missing integrity checks

Assessment Methodology

- + Runtime library behavior monitoring
- + Network traffic analysis for unexpected communications
- + API call monitoring for third-party components
- + Update process analysis

Key Takeaways

- + Modern mobile apps are only as secure as their weakest dependency.
- + Inadequate supply chain controls can result in silent, large-scale compromises.
- + Dynamic analysis must evaluate behavior of all code, even code not written by the developer.

M2: Inadequate Supply Chain Security

Key Takeaways

- + Secure development must include SBOMs, dependency scanning, code signing integrity, and runtime monitoring of 3rd-party components.
- + Mobile red teams and pentesters must profile SDK behavior just as they would a suspected backdoor.

OWASP Mobile Top 10

M3: Insecure Authentication/Authorization

Definitions and Core Concepts

- + **Authentication:** Verifying a user's identity (who you are)
- + **Authorization:** Determining user's access rights (what you can do)
- + **Session Management:** Maintaining and protecting user sessions
- + **Authentication Factors:** Something you know, have, or are
- + **Trust Boundaries:** Where authentication requirements change

Why This Matters in Mobile Contexts

(1/2)

- + Mobile apps introduce unique challenges not seen in traditional web apps:
 - + **Client-Side Logic:** Mobile apps often perform some authZ/authN checks on-device
 - + **Offline Capabilities:** Some apps cache credentials or tokens for offline use

Why This Matters in Mobile Contexts

(2/2)

- + Mobile apps introduce unique challenges not seen in traditional web apps:
 - + **Third-Party SDKs:** OAuth and SSO integrations may not enforce full validation
 - + **Session Persistence:** Long-lived sessions may not require re-authentication
 - + **Local Token Storage:** Tokens often stored insecurely in SharedPreferences, NSUserDefaults, or files

Mobile vs. Web Auth Differences

Concept	Mobile App	Web App
Session Storage	Local storage, SharedPreferences, Keychain	Cookie
Authentication UI	Native (XML/Swift) or WebView	HTML
Token Usage	Often OAuth2, JWT, Firebase tokens	Sessions or cookies
Biometric Auth	Integrated with OS (Keychain, Android Keystore)	Rare
Enforcement Points	Can be split between client & backend	Centralized

Authentication Flaws in Mobile Apps

(1/4)

- + **Weak Credential Management:**
 - + Insecure password policies
 - + Lack of brute force protection
 - + Predictable credential recovery flows
 - + Insecure credential storage
 - + Cleartext credential transmission

Authentication Flaws in Mobile Apps

(2/4)

+ **Broken Client-side Authentication:**

- + Hardcoded credentials
- + Client-side authentication decisions
- + Insecure biometric implementation
- + Bypassing authentication checks
- + Flawed multi-factor authentication

Authentication Flaws in Mobile Apps

(3/4)

- + **Insecure Session Management:**
 - + Insufficient token protection
 - + Missing expiration mechanisms
 - + Vulnerable token generation
 - + Improper session invalidation
 - + Cross-device session handling issues

Authentication Flaws in Mobile Apps

(4/4)

+ **JWT vulnerabilities**

- + Missing signature validation
- + Weak signature algorithms
- + Token information disclosure

+ **OAuth implementation issues**

- + Insecure redirect handling
- + Insufficient state validation
- + Token leakage through logs or URLs

Remember:

- **OAuth** is a protocol for authorization
- **JWT** is a token format

Authorization Flaws in Mobile Apps (1/4)

- + **Insufficient Transport Layer Protection**
 - + Missing or improper TLS implementation
 - + Certificate validation bypasses
 - + Downgrade attacks
 - + Protocol-level vulnerabilities
 - + Insecure communication with backend services

Authorization Flaws in Mobile Apps

(2/4)

- + **Broken Function-level Authorization**
 - + Horizontal privilege escalation (accessing other users' data)
 - + Vertical privilege escalation (accessing higher-privilege functions)
 - + Missing access controls on API endpoints
 - + Insecure direct object references (IDOR)
 - + Race conditions in authorization checks

Authorization Flaws in Mobile Apps

(3/4)

- + **Client-side Authorization Flaws**
 - + Authorization logic in client-side code
 - + Hidden UI elements for privileged functions
 - + Manipulating client-side access control checks
 - + Intent redirection and deep link vulnerabilities
 - + Exported components with insufficient protection

Authorization Flaws in Mobile Apps

(4/4)

- + **Broken Content Provider/Broadcast Receivers Authorization**
 - + Overly permissive content providers
 - + Unprotected broadcast receivers
 - + Intent-based attacks
 - + Insecure data sharing between apps
 - + Insufficient permission checks

Common Dynamic Indicators of M3 Vulnerabilities

(1/2)

- + Login bypass via Frida or modifying API payloads
- + Changing userId or accountId in API requests and receiving valid data
- + Mobile app hiding or showing features based on UI logic — not server enforcement
- + Long-lived session tokens reused across app installs or devices

Common Dynamic Indicators of M3 Vulnerabilities

(2/2)

- + Lack of access token validation in intercepted requests
- + Absence of server-side enforcement for multi-user separation
- + OAuth misconfigurations leaking tokens or enabling authorization code injection

M3: Insecure Authentication/Authorization

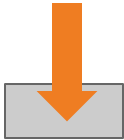
Key Takeaways

- + **Authentication is not just login.** It includes token handling, session lifecycle, and device trust.
- + **Authorization must be server-enforced.** Never rely on mobile client logic for security boundaries.
- + Dynamic testing reveals flaws that static code reviews may miss, such as IDOR and token reuse.
- + Well-configured authentication/authorization systems should resist both client manipulation and backend abuse.

OWASP Mobile Top 10

M4: Insufficient Input/Output Validation

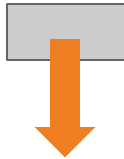
What Is Input/Output Validation?



+

Input Validation:

- + The process of verifying that user-supplied or external data is correct, safe, expected, and adheres to a strict specification before use.



+

Output Validation:

- + Ensuring that data leaving the application is encoded/sanitized appropriately to prevent exploitation of systems that consume it (e.g., web views, APIs, system calls).

Mobile-Specific Input Sources and Attack Vectors (1/5)

- + **User Interface Input Channels:**
 - + Text fields and form data
 - + Gestures and touch events
 - + Voice input and speech recognition
 - + Camera and image input
 - + Biometric input validation considerations

Mobile-Specific Input Sources and Attack Vectors (2/5)

- + **Inter-App Communication Vectors:**
 - + Intent-based communication (Android)
 - + URL schemes and Universal Links (iOS)
 - + App extension interactions
 - + Shared resources and clipboard
 - + Content providers and app-to-app data sharing

Mobile-Specific Input Sources and Attack Vectors (3/5)

- + **Device and Sensor Data:**
 - + Location and GPS data
 - + Accelerometer and gyroscope readings
 - + Bluetooth and NFC inputs
 - + Environmental sensors
 - + Health and fitness data sources

Mobile-Specific Input Sources and Attack Vectors (4/5)

+ **Network and Backend Communications:**

- + API responses and JSON/XML parsing
- + Deep links and dynamic content
- + Push notifications and message handling
- + WebView content and JavaScript interfaces
- + Third-party SDK communications

Mobile-Specific Input Sources and Attack Vectors (5/5)

- + **Platform-Specific Considerations:**
 - + Android Intents vs iOS URL Schemes
 - + Bundle/Extra data validation (Android)
 - + UIPasteboard security (iOS)
 - + Content provider security (Android)
 - + App extension data handling (iOS)

Common Input Validation Vulnerabilities in Mobile Apps

(1/6)

+ Injection Vulnerabilities:

- + SQL Injection in mobile databases
- + NoSQL injection in modern apps
- + XML injection and entity expansion
- + Command injection in native code execution
- + JavaScript injection in WebViews

Common Input Validation Vulnerabilities in Mobile Apps

(2/6)

+ **Cross-Site Scripting (XSS) in Mobile Context:**

- + Stored XSS in mobile clients
- + Reflected XSS in WebViews
- + DOM-based XSS vulnerabilities
- + JavaScript bridges exploitation
- + Template injection vulnerabilities

Common Input Validation Vulnerabilities in Mobile Apps

(3/6)

- + **Input Validation Bypass Techniques:**
 - + Encoding and special character handling
 - + Input length manipulation attacks
 - + MIME type and content type spoofing
 - + JSON and XML structural attacks
 - + Multi-stage validation bypass techniques

Common Input Validation Vulnerabilities in Mobile Apps

(4/6)

- + **File and Media Input Vulnerabilities:**
 - + Path traversal in file operations
 - + Insecure file handling and storage
 - + Media parsing vulnerabilities and exploits
 - + File type validation bypasses
 - + Malicious document handling risks

Common Input Validation Vulnerabilities in Mobile Apps

(5/6)

+ Intent-Based Vulnerabilities (Android):

- + Intent spoofing attacks
- + Unvalidated Intent extras
- + Implicit Intent interception
- + PendingIntent misuse
- + Activity hijacking through Intents

Common Input Validation Vulnerabilities in Mobile Apps

(6/6)

- + **URL Scheme Vulnerabilities (iOS):**
 - + URL scheme hijacking
 - + Parameter pollution in URL schemes
 - + Universal Link validation issues
 - + Deep link redirection vulnerabilities
 - + URL scheme enumeration techniques

Output Validation and Encoding Vulnerabilities

(1/4)

- + **Output Context Considerations:**
 - + UI rendering contexts (HTML, XML, Native)
 - + JSON/XML data serialization issues
 - + Logging output security
 - + File output validation
 - + Inter-app communication output

Output Validation and Encoding Vulnerabilities (2/4)

- + **Insufficient Output Encoding:**
 - + HTML entity encoding failures
 - + JavaScript encoding in WebViews
 - + SQL value encoding for queries
 - + XML output encoding issues
 - + Character set and encoding inconsistencies

Output Validation and Encoding Vulnerabilities

(3/4)

+ **Content Security Policy in Mobile Context:**

- + WebView CSP implementation
- + Limitations of CSP in hybrid apps
- + Native code vs web content boundaries
- + CSP bypass techniques specific to mobile
- + Best practices for mobile CSP

Output Validation and Encoding Vulnerabilities

(4/4)

+ Information Leakage through Output:

- + Error message exposure
- + Stack traces and debug information
- + Verbose logging of sensitive data
- + Metadata leakage in output files
- + Caching of sensitive output data

Testing Methodologies for Input/Output Validation

- + Input fuzzing methodologies for mobile
- + Intercepting and manipulating mobile traffic
- + Runtime validation bypass detection
- + Instrumenting apps to monitor validation
- + Mobile-specific fuzzing tools and approaches

Attack Surface for Input/Output Vectors in Mobile (1/2)

- + **UI input fields:** Forms, login boxes, profile editors
- + **Deep links & URI schemes:** myapp://login?token=...
- + **External files or content providers:** Uploaded PDFs, XML, JSON
- + **Intents and Inter-Process Communication (IPC):** Explicit or implicit intents from other apps
- + **WebViews:** Rendering of HTML or unescaped strings

Attack Surface for Input/Output Vectors in Mobile (2/2)

- + **Sensor data:** GPS spoofing, microphone input, gyroscope
- + **Cloud sync:** Syncing tampered offline data to server
- + **Clipboard data:** Pasted input from untrusted sources
- + **Logs and error messages:** Exposing sensitive or manipulated values

Dynamic Indicators of M4 Vulnerabilities

(1/2)

- + App crashes or behaves erratically with crafted input: Input parsing flaws
- + Data echoed back in UI, logs, or notifications: Reflected output injection
- + URL or file paths constructed using user-controlled input: Path traversal, SSRF, insecure file access

Dynamic Indicators of M4 Vulnerabilities

(2/2)

- + WebView rendering dynamic input: DOM injection, local XSS
- + Unfiltered input sent to backend APIs: Remote injection possibilities
- + Input accepted from URI, NFC, QR, BLE, or sensors: Sensor/input spoofing or injection channels

M4: Insufficient Input/Output Validation

Key Takeaways

- + **Input/output is not just about text boxes.**
- + **Untrusted input can arrive from anywhere.** Sensors, NFC, QR codes, deep links, and even clipboard data can be attack vectors.
- + **Output injection is just as dangerous.** Especially in WebViews, logs, or reflected UI strings.
- + **Mobile attackers focus on chaining input validation flaws** to escalate privileges or extract data — dynamic analysis is key to spotting them.

OWASP Mobile Top 10

M5: Insecure Communication

What Is Insecure Communication?

- + **Mobile applications sometimes fail to adequately protect data in transit between the mobile device and remote systems such as:**
 - + Application servers
 - + Cloud services
 - + APIs
 - + Third-party services (ads, analytics, auth)
 - + Local systems (IoT, BLE, NFC)

What Is a Certificate?

- + A digital certificate is a file that proves the identity of a server.
- + It is issued by a Certificate Authority (CA).
- + Contains:
 - + Public key
 - + Domain name
 - + Issuer information
 - + Expiration date
 - + Digital signature

Why Certificates Matter in Mobile Apps?

- + Mobile apps often use untrusted networks (public Wi-Fi, mobile data).
- + Without proper TLS certificate validation:
 - + MITM attacks become trivial
 - + Data can be intercepted or modified
 - + Tokens, PII, and credentials are at risk

Common Certificate Handling Mistakes

- + Accepting all certificates (no validation)
- + Allowing self-signed certificates
- + Disabling hostname verification
- + Using expired or weak certificates
- + Forgetting to re-enable validation in production builds

How Certificate Validation Works?

1. App connects to server via HTTPS
2. Server presents its certificate
3. App verifies:
 - a. Issuer is trusted
 - b. Domain matches
 - c. Certificate is valid (not expired/revoked)
4. Secure session is established

What Is Certificate Pinning?

- + Pinning = trusting only a specific certificate or public key.
- + Prevents abuse of rogue CAs.
- + Even if an attacker has a valid cert from a CA, the app won't trust it.
- + Can be implemented via:
 - + Hardcoded certs/public keys
 - + Android Network Security Config
 - + iOS ATS + SSL pinning libraries

Pinning vs. No Pinning

Scenario	Pinning	No Pinning
MITM with rogue CA	Rejected	Accepted
Legit cert update	App update needed	Accepted
Pinning misconfiguration	Breaks connection	N/A

Dynamic Indicators of Insecure Communication

- + App traffic visible in Burp/mitmproxy without effort: No certificate pinning
- + Credentials or tokens sent via HTTP: No encryption at all
- + Unexpected domains: Third-party SDKs leaking data
- + Auth tokens in query strings: Replay risk, leaks in logs
- + TLS errors absent with invalid certs: No cert validation
- + Plaintext values in app logs: No output sanitization

Attack Techniques

(1/2)

- + Man-in-the-Middle (MITM): Intercept app traffic using proxy (Burp Suite, mitmproxy)
- + TLS Downgrade Attack: Force app to use older TLS or fallback to HTTP
- + Fake Certificate Injection: Use a self-signed cert on a rooted device/emulator

Attack Techniques

(2/2)

- + DNS Spoofing: Redirect app to attacker's server
- + Protocol Analysis: Examine API and BLE protocols for replay or injection
- + Token/Session Hijacking: Steal access tokens via unprotected endpoints

M5: Insecure Communication

Key Takeaways

- + Insecure communication is the gateway to full compromise
- + Mobile devices frequently operate in hostile networks
- + Cert validation and pinning are your first lines of defense
- + Dynamic analysis tools like Burp, mitmproxy, and Frida reveal how secure your traffic really is
- + Every app should treat network traffic as a critical asset requiring the same protection as local data storage.

OWASP Mobile Top 10

M6: Inadequate Privacy Controls

What Are Privacy Controls?

- + Mechanisms and policies designed to limit the collection, usage, storage, and sharing of personal or sensitive data in a way that complies with laws, user expectations, and security best practices

Regulatory Framework

- + GDPR and its impact on mobile applications
- + CCPA/CPRA requirements for mobile apps
- + PIPEDA (Canada) and international regulations
- + App store privacy requirements (Apple & Google)
- + Industry-specific regulations (HIPAA, GLBA, etc.)
- + Children's privacy (COPPA) in mobile context

What Qualifies as Personal or Sensitive Data?

- + Personal Identifiers
- + Account Credentials
- + Financial Data
- + Biometric Data / Health Data
- + Device Identifiers
- + Location Data
- + Behavioral Data

Types of Privacy Concerns in Mobile Applications

(1/5)

- + **Data Collection and Minimization:**
 - + Excessive data collection practices
 - + Just-in-time collection vs. preemptive collection
 - + Unnecessary permissions and data access
 - + Context-appropriate data collection
 - + Data minimization strategies

Types of Privacy Concerns in Mobile Applications

(2/5)

- + **Identification and Tracking:**
 - + Device fingerprinting techniques
 - + Advertising identifiers (IDFA, AAID)
 - + Cross-app tracking mechanisms
 - + Location tracking precision issues
 - + Persistent identifiers and linkability

Types of Privacy Concerns in Mobile Applications

(3/5)

- + **Data Retention and Deletion:**
 - + Inappropriate data retention periods
 - + Incomplete data deletion
 - + Data persistence across app reinstalls
 - + Backup and sync privacy implications
 - + Cloud vs. device-only storage considerations

Types of Privacy Concerns in Mobile Applications

(4/5)

+ **Data Sharing and Third-Party Integration:**

- + Third-party SDK privacy implications
- + Analytics platforms and data collection
- + Ad networks and tracking libraries
- + Social media integration privacy concerns
- + Cross-application data sharing

Types of Privacy Concerns in Mobile Applications

(5/5)

- + **Informed Consent and Transparency:**
 - + Effective notice and consent mechanisms
 - + Dark patterns in privacy interfaces
 - + Readability of privacy policies
 - + Just-in-time privacy notices
 - + User control over privacy settings

Why Privacy Is Different in Mobile Apps?

- + Sensor access
- + Background data collection
- + Permission abuse
- + Third-party SDKs
- + Unregulated data flow
- + Cross-app data access

Common Privacy Vulnerabilities in Mobile Applications

(1/6)

+ Excessive Permission Usage:

- + Camera and microphone access beyond necessity
- + Background location tracking
- + Contact list over-access
- + Storage access beyond requirements
- + Runtime permission bypasses

Common Privacy Vulnerabilities in Mobile Applications

(2/6)

+ Insecure Data Transmission:

- + Privacy implications of unencrypted transmission
- + Metadata leakage in API communications
- + Headers exposing sensitive information
- + Third-party API privacy concerns
- + Analytics data transmission security

Common Privacy Vulnerabilities in Mobile Applications

(3/6)

+ **Persistent Identifiers and Device Fingerprinting:**

- + Hardware identifiers (IMEI, MAC addresses)
- + Application-generated persistent IDs
- + Canvas fingerprinting in WebViews
- + Audio and sensor-based fingerprinting
- + Fingerprinting resistance techniques

Common Privacy Vulnerabilities in Mobile Applications

(4/6)

+ **Insufficient Anonymization/Pseudonymization:**

- + Failed anonymization techniques
- + Re-identification risks
- + Aggregation and k-anonymity failures
- + Location data pseudonymization challenges
- + Temporal data correlation risks

Common Privacy Vulnerabilities in Mobile Applications

(5/6)

- + **Privacy-Invasive Analytics:**
 - + Excessive session recording
 - + Keylogging and input tracking
 - + User behavior surveillance
 - + Heatmaps and interaction recording
 - + Analytics data sent to multiple parties

Common Privacy Vulnerabilities in Mobile Applications

(6/6)

+ Metadata Leakage:

- + Image EXIF data exposure
- + Document metadata retention
- + Communication metadata collection
- + Telemetry data privacy implications
- + Debugging logs with sensitive information

Dynamic Testing Indicators of Privacy Issues (1/2)

- + Access to camera or mic on app start: Background surveillance or sensor abuse
- + GPS data sent frequently: Location tracking, often without reason
- + Unexpected domains in Burp Suite: Third-party SDKs leaking data

Dynamic Testing Indicators of Privacy Issues

(2/2)

- + Logs or local files contain PII: Data exposure risk on stolen/lost devices
- + No permission dialogs or vague ones: Poor consent and transparency
- + Session tokens, emails in crash reports: Logging sensitive info by mistake

Testing for Privacy Vulnerabilities (1/4)

- + **Dynamic Privacy Analysis:**
 - + Network traffic analysis for privacy leaks
 - + Data flow tracking methodologies
 - + Proxy-based privacy testing
 - + Monitoring filesystem for data persistence
 - + Tracking data access events

Testing for Privacy Vulnerabilities (2/4)

+ Behavioral Analysis:

- + App behavior in different privacy contexts
- + Testing for privacy policy compliance
- + Comparing claimed vs. actual data practices
- + Privacy control effectiveness testing
- + Background behavior monitoring

Testing for Privacy Vulnerabilities (3/4)

- + **Data Mapping and Inventory:**
 - + Identifying data collection points
 - + Data flow mapping through application
 - + Storage location inventory
 - + Third-party data sharing mapping
 - + Data lifecycle tracing

Testing for Privacy Vulnerabilities (4/4)

+ **Consent and Transparency Testing:**

- + Evaluating notice quality and timing
- + Consent withdrawal testing
- + Testing for dark patterns
- + Privacy settings effectiveness
- + Accessibility of privacy controls

M6: Inadequate Privacy Controls

Key Takeaways

(1/2)

- + **Privacy ≠ Security.** You can have strong security controls but still violate user privacy.
- + **Dynamic testing often reveals hidden privacy violations** from SDKs or background activity.
- + **Mobile platforms grant powerful data access** — developers must handle that responsibility carefully.

M6: Inadequate Privacy Controls

Key Takeaways

(2/2)

- + **Penetration testers should evaluate privacy flows** just like they test for technical vulnerabilities.
- + **The real enemy is invisibility:** hidden data leaks often go unnoticed until exposed by threat actors or regulators.

OWASP Mobile Top 10

M7: Insufficient Binary Protections

What Are Binary Protections?

- + Binary protections are defensive techniques applied to the compiled application package (APK for Android, IPA for iOS) to prevent or hinder reverse engineering, tampering, and dynamic analysis.

What Are Binary Protections?

- + They make it harder to:
 - + Decompile the app
 - + Understand internal logic
 - + Modify security checks
 - + Inject or hook malicious code
 - + Extract secrets or credentials

Why Binary Protection Matters in Mobile Apps?

(1/2)

- + **Mobile apps are easily accessible:** Anyone can download, decompile, and analyze them
- + **Client-side logic is visible:** If not protected, code reveals secrets, logic, tokens
- + **Tampering is common:** Apps can be patched, repackaged, or wrapped with malware

Why Binary Protection Matters in Mobile Apps?

(2/2)

- + **Repackaging threats:** Modified apps may be redistributed with added malware
- + **Runtime hooking:** Without protections, attackers can inject code dynamically using Frida, Xposed, etc.

Myths vs. Reality

- + “Obfuscation = Security”
- + “Nobody will bother decompiling our app”
- + “Our APIs are protected”
- + “Anti-root = secure”

Dynamic Analysis Methods

- + Runtime binary manipulation
- + Memory dumping techniques
- + Method hooking/swizzling
- + Function call interception
- + Native library injection

Dynamic Assessment Techniques

- + Runtime protection testing
- + Debugger attachment attempts
- + Hook detection bypass testing
- + Memory analysis techniques
- + Runtime manipulation attempts
- + Instrumentation resistance evaluation

Protection Bypass Methods

- + De-obfuscation techniques
- + Dynamic instrumentation with Frida
- + Runtime memory modification
- + Hook detection evasion
- + Emulator detection bypass
- + Binary patching approaches

M7: Insufficient Binary Protections

Key Takeaways

(1/2)

- + Binary protections don't eliminate vulnerabilities — they make exploitation harder.
- + Attackers use well-known tooling like jadx and Frida to extract secrets, bypass logic, or patch apps.
- + Obfuscation, anti-debugging, and runtime checks create friction for attackers — use them in combination.

M7: Insufficient Binary Protections

Key Takeaways

(2/2)

- + Dynamic mobile testing must evaluate how resilient an app is under reverse engineering, tampering, and instrumentation.
- + Security must not rely on the client alone — protect the server side even if the app is compromised.

OWASP Mobile Top 10

M8: Security Misconfiguration

What Is a Security Misconfiguration?

- + Security settings are left in their default state
- + Dangerous features are left enabled in production
- + Insecure values are set in config files, manifests, or entitlements
- + Permissions are overly broad, exposing data or attack surfaces
- + The app or server is incompletely hardened or untested

Why Misconfiguration is a Big Deal in Mobile

- + Mobile apps are shipped with configuration files
- + Many devs use debug settings in production
- + Platform APIs are powerful and complex
- + Apps bundle third-party SDKs
- + App behavior can change across environments

Common Mobile Misconfiguration Vulnerabilities

(1/5)

- + **Excessive Application Permissions:**
 - + Over-privileged applications
 - + Permission justification and necessity
 - + Dangerous permission combinations
 - + Permission escalation through misconfiguration
 - + Runtime vs. install-time permission issues

Common Mobile Misconfiguration Vulnerabilities

(2/5)

- + **Insecure Communication Settings:**
 - + Missing network security configurations
 - + Certificate validation disabling
 - + Cleartext traffic allowances
 - + Insecure TLS configurations
 - + Mixed content allowances in WebViews

Common Mobile Misconfiguration Vulnerabilities

(3/5)

- + **Data Storage Misconfigurations:**
 - + World-readable/writable files and directories
 - + Insecure external storage usage
 - + Unprotected SQLite databases
 - + Insecure shared preferences settings
 - + Caching and temporary file misconfigurations
 - + Accessibility of sensitive app data

Common Mobile Misconfiguration Vulnerabilities (4/5)

- + **Build and Deployment Misconfigurations:**
 - + Debug flags left enabled
 - + Insecure compiler options
 - + Developer backdoors remaining in production
 - + Test endpoints exposed in production builds
 - + Excessive logging and exposure of sensitive information
 - + Signing certificate misconfigurations

Common Mobile Misconfiguration Vulnerabilities

(5/5)

- + **Component Exposure Issues:**
 - + Unprotected Activity components
 - + Insecure Service implementations
 - + Broadcast Receiver vulnerabilities
 - + Content Provider access control issues
 - + Deep link handling misconfigurations
 - + Exported component risks

Testing for Security Misconfigurations (1/3)

- + **Dynamic Configuration Testing:**
 - + Runtime permission behavior testing
 - + Component accessibility testing
 - + Inter-app communication testing
 - + File system permission validation
 - + Data storage configuration verification
 - + Network configuration validation

Testing for Security Misconfigurations

(2/3)

+ **Configuration Testing Tools and Techniques:**

- + MobSF for configuration analysis
- + Drozer for Android component testing
- + Mobile app property extraction
- + Network configuration testing tools
- + Automated configuration scanning
- + Custom configuration test script development

Testing for Security Misconfigurations

(3/3)

- + **Backend Configuration Testing**
 - + API endpoint security validation
 - + Authentication configuration testing
 - + Mobile-specific backend configuration review
 - + Cloud service configuration validation
 - + Mobile API gateway testing
 - + Push notification service configuration testing

M8: Security Misconfiguration

Key Takeaways

- + Misconfigurations are low-effort, high-impact vulnerabilities that attackers exploit routinely.
- + Many of them are visible statically, but validated dynamically
- + Common developer mistakes can lead to major security gaps.
- + Proper hardening and testing of all environments and configurations is a must before going to production.

OWASP Mobile Top 10

M9: Insecure Data Storage

What Is Insecure Data Storage?

- + Sensitive information being stored locally on the mobile device in a way that is inadequately protected, allowing attackers to read, manipulate, or exfiltrate it, especially on compromised (rooted/jailbroken) devices.

Why This Matters in Mobile Apps?

- + Mobile devices are personal, physical, and often lost/stolen
- + Apps that work offline require caching or storing data locally
- + Users rarely encrypt their devices
- + Root/jailbreak tooling is common among attackers
- + Data often synced from device to cloud

Common Insecure Data Storage Vulnerabilities

(1/4)

- + **Storage of Sensitive Data in Plaintext:**
 - + Credentials in clear text
 - + Personal Identifiable Information (PII) exposure
 - + Payment information in unprotected storage
 - + Health and medical data exposure
 - + Authentication tokens in clear text
 - + Geolocation data in unprotected formats

Common Insecure Data Storage Vulnerabilities (2/4)

+ Insecure Data Storage Locations:

- + World-readable files and directories
- + External storage usage for sensitive data
- + Unprotected database files
- + Temporary files with sensitive content
- + Cache directories containing sensitive data
- + Insecure backup storage

Common Insecure Data Storage Vulnerabilities

(3/4)

- + **Insecure Data Sharing Mechanisms:**
 - + Content provider misconfigurations
 - + Intent-based data sharing vulnerabilities
 - + Pasteboard data exposure
 - + URL scheme data leakage
 - + Document provider vulnerabilities
 - + Cross-application data exposure

Common Insecure Data Storage Vulnerabilities

(4/4)

- + **Data Persistence Issues:**
 - + Excessive data retention
 - + Incomplete data deletion
 - + Data persistence after uninstallation
 - + Improper cache management
 - + Insecure crash dumps and logs
 - + Application backup vulnerabilities

Dynamic Exploitation of Insecure Data Storage

(1/4)

- + **Accessing Android App Data Storage:**
 - + Exploring app data directory structure
 - + Accessing Shared Preferences
 - + Extracting and analyzing SQLite databases
 - + Examining file permissions and access
 - + Analyzing XML storage files
 - + Automated tools for Android data extraction

Dynamic Exploitation of Insecure Data Storage

(2/4)

- + **Accessing iOS App Data Storage:**
 - + iOS application sandbox structure
 - + Extracting and analyzing plist files
 - + Accessing Core Data and Realm databases
 - + Determining file permissions and protection classes
 - + Keychain extraction techniques
 - + iOS backup analysis

Dynamic Exploitation of Insecure Data Storage

(3/4)

- + **Runtime Data Analysis Techniques:**
 - + Memory dumping and analysis
 - + Runtime data interception
 - + Method hooking for data access
 - + Dynamic instrumentation of storage APIs
 - + Live application manipulation
 - + CPU register examination for sensitive data

Dynamic Exploitation of Insecure Data Storage (4/4)

+ **Data Storage Side-Channel Attacks:**

- + Timing analysis of storage operations
- + Power analysis on mobile devices
- + Cache-based side channels
- + Storage-related covert channels
- + Forensic analysis of storage artifacts
- + Memory residue analysis

Testing for Insecure Data Storage

- + Filesystem monitoring during app operation
- + Storage API call interception
- + Runtime data manipulation
- + Backup analysis techniques
- + Database extraction and inspection
- + Preference and settings extraction

Common Storage Locations

(1/2)

- + SharedPreferences / NSUserDefaults (Android / iOS): May store unencrypted tokens, session data, settings
- + Internal App Storage (Android: /data/data/): May be accessible via root
- + External Storage (SD card / Documents folder): World-readable, accessible by any app
- + SQLite Databases / CoreData (Android / iOS): Unencrypted by default

Common Storage Locations

(2/2)

- + Log Files: PII or credentials leaked into runtime or crash logs
- + Clipboard: Sensitive data temporarily stored and exposed
- + Keychain / Keystore misuse (iOS / Android): Storing in insecure contexts or with weak access controls
- + Backups iOS (iTunes/iCloud), Android (ADB backup)

Dynamic Indicators of Insecure Storage

- + Login persists after app reinstall
- + Crash logs contain usernames/tokens
- + Clipboard captures app data
- + adb backup shows sensitive data
- + App saves downloaded reports, invoices in /sdcard/ (World Readable)

M9: Insecure Data Storage

Key Takeaways

- + Client-side data is never fully safe — it must be intentionally secured.
- + Attackers don't need your app's source code — they just need your app binary and access to storage.
- + Dynamic testing is essential to find what's really being stored during runtime.
- + **Always ask:** "If this device is lost, rooted, or infected — what will be exposed?"

OWASP Mobile Top 10

M10: Insufficient Cryptography

What Is Insufficient Cryptography ?

- + Using broken or weak algorithms (e.g., MD5, RC4, ECB mode)
- + Incorrect implementation of standard algorithms (e.g., no IV, static salt)
- + Using custom-built cryptographic logic (“homegrown crypto”)
- + Weak key generation or hardcoded keys
- + Not encrypting sensitive data when it should be

Common Cryptographic Vulnerabilities in Mobile Applications

(1/4)

- + **Weak Cryptographic Algorithms:**
 - + Usage of obsolete algorithms (MD5, SHA-1, DES)
 - + Insufficient key lengths
 - + ECB mode usage for symmetric encryption
 - + Non-standard or proprietary algorithms
 - + Quantum vulnerability considerations
 - + Deprecated cryptographic primitives

Common Cryptographic Vulnerabilities in Mobile Applications

(2/4)

+ **Improper Cryptographic Implementation:**

- + Hardcoded cryptographic keys
- + Predictable initialization vectors (IVs)
- + Insufficient key derivation
- + Lack of authenticated encryption
- + Poor random number generation
- + Missing salt in password hashing
- + Padding oracle vulnerabilities

Common Cryptographic Vulnerabilities in Mobile Applications

(3/4)

- + **Key Management Vulnerabilities:**
 - + Insecure key storage
 - + Inadequate key rotation
 - + Shared keys across installations
 - + Key extraction vulnerabilities
 - + Weak key derivation functions
 - + Insecure key transmission
 - + Missing key revocation mechanisms

Common Cryptographic Vulnerabilities in Mobile Applications

(4/4)

+ **Cryptographic Oracles and Side Channels:**

- + Timing attack vulnerabilities
- + Error message information leakage
- + Cache-based cryptographic attacks
- + Power analysis vulnerabilities
- + Implementation-based side channels
- + Padding oracle attacks in mobile context

Dynamic Analysis of Cryptographic Operations

- + Runtime cryptographic function monitoring
- + Key extraction attempts
- + Cryptographic API interception
- + Memory analysis for cryptographic material
- + Side-channel analysis techniques
- + Cryptographic oracle testing

Testing Cryptographic Key Management

- + Key storage security assessment
- + Key generation validation
- + Key transmission security testing
- + Key lifecycle management review
- + Hardware key protection verification
- + Backup and recovery of cryptographic keys

Cryptography in Dynamic Testing (1/2)

- + Data decrypted easily via memory dump: Weak or static keys used
- + Plaintext values written to disk or logs: Crypto not applied where needed
- + Predictable ciphertext: ECB mode or no random IV used
- + Same output for same input: Lack of randomness (e.g., no salt)

Cryptography in Dynamic Testing

(2/2)

- + Decryption possible with Frida or Objection: Keys extracted at runtime or weak encryption
- + Custom logic found in decompiled code: Homegrown crypto vulnerable to reversal
- + Crypto implementation crashes under fuzzing: Poor error handling or key management

M10: Insufficient Cryptography

Key Takeaways

- + Weak crypto is worse than no crypto
- + Cryptographic flaws can be detected dynamically via traffic, memory, and runtime analysis.
- + Hardcoded secrets are gold for attackers.
- + Modern devices offer strong crypto tools — the real issue is correct usage.

Mapping All Mobile Tests to MASVS and Real-World Attack Chains

Why This Module Is Critical?

- + Strategic Testing: You should know what each test proves, not just how to run it
- + Adversarial Mindset: Every test maps to attacker tactics: credential theft, privilege escalation, lateral movement
- + Defense Engineering: Tests must map to defense-in-depth and threat modeling frameworks

Mobile Application Security Verification Standard (MASVS) (1/2)

- + MASVS-ARCH: Architecture & threat modeling
- + MASVS-RESILIENCE: Reverse engineering & tampering protections
- + MASVS-STORAGE: Local data storage protections
- + MASVS-CRYPTO: Secure usage of cryptographic APIs

Mobile Application Security Verification Standard (MASVS) (2/2)

- + MASVS-AUTH: Authentication & session management
- + MASVS-NETWORK: Network security (TLS, cert pinning, session tokens)
- + MASVS-PLATFORM: Secure use of mobile OS features
- + MASVS-CODE: Code quality, hardening, debugging

Key Takeaways

- + **Every test should serve a strategic goal** — either defending against known attacker behavior or validating a MASVS requirement.
- + **The power of MASVS is not in checking boxes**, but in designing and defending apps against real-world threat actors.
- + **A mobile pentester is not just a scanner operator** — they're a threat modeler, chain-builder, and story-teller.

A person wearing glasses is shown from a high angle, looking down at a laptop. The scene is illuminated with a strong blue and purple light, creating a tech-oriented atmosphere. The person's hands are visible on the laptop keyboard.

Android Dynamic Testing

Dynamic vs. Static Testing Approaches

(1/2)

- + Dynamic testing offers several advantages over static analysis:
 - + **Runtime behavior observation:** You can see exactly how the application functions during execution
 - + **State-dependent vulnerabilities:** Some issues only appear based on specific application states

Dynamic vs. Static Testing Approaches

(2/2)

- + **Bypassing obfuscation:** Dynamic testing can work around code obfuscation that might hinder static analysis
- + **Real-world exploitation:** You can verify vulnerabilities by exploiting them in their actual environment
- + *“I once spent days trying to statically analyze an obfuscated financial app with no success. When I switched to dynamic analysis, I found three critical vulnerabilities in the first hour by simply observing the app's behavior during specific transactions”*

Android Security Architecture Relevant to Dynamic Testing

(1/2)

- + To effectively test Android applications, you need to understand several key components of the Android security model:
 - + **Application sandboxing:** Each app runs in its own process with a unique user ID
 - + **Permission model:** Apps must request permissions to access protected APIs and resources
 - + **Component exposure:** Activities, services, broadcast receivers, and content providers can be exposed to other applications



Android Security Architecture Relevant to Dynamic Testing (2/2)

- + To effectively test Android applications, you need to understand several key components of the Android security model:
 - + **IPC mechanisms:** Intents, AIDL, binders, and other mechanisms that facilitate inter-process communication
 - + **WebView security:** JavaScript interfaces that bridge web and native code
 - + **Network security:** Certificate pinning, encryption, and TLS implementation

The Testing Environment

- + Rooted Android device or emulator
- + Android debugging tools (ADB)
- + Interception proxy (Burp Suite)
- + Runtime manipulation tools (Frida)
- + Android analysis tools (Drozer, MobSF)
- + Network analysis tools (Wireshark)

Bypassing Root Detection

(1/6)

- + Root detection methods typically fall into several categories:
 - + File-based checks: Looking for SU binary, Superuser.apk, etc.
 - + Property checks: Examining system properties
 - + Package-based checks: Detecting root management apps
 - + Permission checks: Testing for root permissions
 - + Running commands with su

Bypassing Root Detection (2/6)

- + Root detection methods typically fall into several categories:
 - + File-based checks: Looking for SU binary, Superuser.apk, etc.

```
public boolean checkForSUBinary() {  
    String[] paths = {"/system/bin/su", "/system/xbin/su", "/sbin/su"};  
    for (String path : paths) {  
        if (new File(path).exists()) return true;  
    }  
    return false;  
}
```


Bypassing Root Detection

(3/6)

- + Root detection methods typically fall into several categories:
 - + Property checks: Examining system properties

```
public boolean checkRoProperties() {  
    String roKeys[] = {"ro.secure", "ro.debuggable"};  
    String roValues[] = {"0", "1"};  
  
    for (int i = 0; i < roKeys.length; i++) {  
        String value = android.os.SystemProperties.get(roKeys[i]);  
        if (value.equals(roValues[i])) return true;  
    }  
    return false;  
}
```

Bypassing Root Detection (4/6)

- + Root detection methods typically fall into several categories:
 - + Package-based checks: Detecting root management apps

```
public boolean checkForRootManagementApps() {  
    String[] rootApps = {"com.noshufou.android.su", "com.thirdparty.superuser"};  
    PackageManager pm = getPackageManager();  
    for (String rootApp : rootApps) {  
        try {  
            pm.getPackageInfo(rootApp, 0);  
            return true;  
        } catch (PackageManager.NameNotFoundException e) {}  
    }  
    return false;  
}
```

Bypassing Root Detection (5/6)

- + Root detection methods typically fall into several categories:
 - + Permission checks: Testing for root permissions

```
public boolean checkForRWPaths() {  
    String[] paths = {"/system", "/system/bin"};  
    for (String path : paths) {  
        File file = new File(path);  
        if (file.canWrite()) return true;  
    }  
    return false;  
}
```

Bypassing Root Detection (6/6)

- + Root detection methods typically fall into several categories:
 - + Running commands with su

```
public boolean checkSuExists() {  
    Process process = null;  
    try {  
        process = Runtime.getRuntime().exec(new String[] {"which", "su"});  
        BufferedReader reader = new BufferedReader(new  
        InputStreamReader(process.getInputStream()));  
        return reader.readLine() != null;  
    } catch (Exception e) {  
        return false;  
    } finally {  
        if (process != null) process.destroy();  
    }  
}
```

Why Certificate Pinning Matters?

- + Standard certificate validation only verifies trust chain (CA signature)
- + Any CA-signed certificate is accepted by default, even from different CAs
- + Over 150+ root CAs in typical trust stores - all are implicitly trusted
- + Compromised CA could issue fraudulent certificates
- + Pinning restricts trust to specific expected certificates only

Certificate Pinning: Technical Implementation

```
OkHttpClient client = new OkHttpClient.Builder()  
    .certificatePinner(new CertificatePinner.Builder()  
        .add("api.example.com",  
            "sha256:NxNDNfbK28R82UzDQ9iqbT87uWJBnL6h1WGYyeWGVpg=")  
        .add("api.example.com",  
            "sha256:6WCELdG63gA9WSWQMmSUXTC9WJjkpr2Fh1jMDHG3+rw=")  
        .build())  
    .build();
```

Certificate Pinning Methods

(1/3)

- + **Pin the Certificate:**
 - + Stores the entire certificate
 - + Most restrictive approach
 - + Must be updated when certificate expires

Certificate Pinning Methods

(2/3)

- + **Pin the Public Key:**
 - + Stores the public key hash
 - + Survives certificate renewal (if same key pair is used)
 - + More flexible than certificate pinning

Certificate Pinning Methods

(3/3)

- + **Pin the CA Certificate:**
 - + Pins the issuing Certificate Authority
 - + Most flexible but less secure
 - + Allows any certificate from that CA

Why Bypass During Testing?

- + Intercept API traffic for security assessment
- + Analyze data transmission for sensitive information
- + Verify proper implementation of backend security controls
- + Test for vulnerabilities in API endpoints

Common Bypass Techniques

- + Dynamic instrumentation (Frida, Objection)
- + Xposed modules (JustTrustMe, SSLUnpinning)
- + Manual code modification (smali/Java patching)
- + Application repackaging

Advanced Pinning

(1/2)

- + **Multi-Layered Pinning:**
 - + Applications may implement multiple verification mechanisms
 - + Certificate checks at different levels (native, Java)
 - + Verification at app startup and during requests

Advanced Pinning

(1/2)

- + **Multi-Layered Pinning:**
 - + Applications may implement multiple verification mechanisms
 - + Certificate checks at different levels (native, Java)
 - + Verification at app startup and during requests

Advanced Pinning (2/2)

- + **Bypass Detection Mechanisms:**
 - + Integrity checks to detect tampering
 - + Anti-hooking/anti-Frida detection
 - + Emulator/root detection combined with pinning

Tampering Runtime Logic with Frida Scripts

- + Frida uses a client-server architecture:
 - + The Frida server runs on the target device
 - + The Frida client runs on your machine and communicates with the server
 - + JavaScript is injected into target processes to manipulate their behavior

Key Frida capabilities

- + Hooking functions and methods
- + Reading and modifying memory
- + Tracing execution
- + Enumerating loaded classes and modules
- + Calling functions directly
- + Manipulating Java/Kotlin/Objective-C objects

A high-angle, close-up shot of a person with dark hair and glasses, looking down at a laptop. The scene is dimly lit with a strong blue and purple color cast, likely from the laptop screen and ambient lighting. The person's hands are visible on the keyboard. The background is dark and out of focus.

iOS Application Dynamic Testing



API & Backend Testing for Mobile Apps

What Are APIs?

- + API = Application Programming Interface
- + Acts as a bridge between mobile apps and backend services
- + Enables data exchange, feature execution, and remote interaction
- + Fundamental to both Android and iOS development
- + **83% of mobile app traffic is API-driven** (Gartner, 2023)

Why APIs Are Critical in Mobile Apps?

- + Offloads heavy processing to the backend (authentication, business logic)
- + Enables real-time data updates (e.g., messaging, notifications)
- + Keeps the app lightweight and responsive
- + Supports scalability and central data management

Common Use Cases for APIs in Mobile Apps

- + Login & authentication (OAuth, Firebase Auth)
- + Profile & user settings retrieval
- + Data syncing (chat, tasks, shopping carts)
- + Payment processing (Stripe, Apple Pay APIs)
- + Push notifications (FCM/APNs)

Multiple API Layers Power Mobile Experiences (1/2)

- + Backend APIs:
 - + RESTful APIs (most common in mobile apps),
 - + GraphQL (gaining popularity for data efficiency),
 - + gRPC, WebSockets, etc.
- + Platform-Specific APIs:
 - + iOS: Apple frameworks (UIKit, SwiftUI, Core Location, HealthKit)
 - + Android: Google Play Services, Jetpack libraries, Firebase



Multiple API Layers Power Mobile Experiences (2/2)

- + Third-Party APIs:
 - + Payment processors (Stripe, PayPal)
 - + Social media integration (Facebook, Twitter)
 - + Maps and location services (Google Maps, Mapbox)
 - + Analytics platforms (Firebase Analytics, Mixpanel)

API Security Challenges in Mobile Environments

(1/3)

- + Client-Side Exposure:
 - + API endpoints and authentication mechanisms in mobile code
 - + Reverse engineering risks compared to web applications
 - + Greater persistence of authentication tokens

API Security Challenges in Mobile Environments

(2/3)

- + Platform-Specific Vulnerabilities:
 - + Android: Deep linking and intent-based attacks
 - + iOS: URL scheme manipulation and shared keychain issues

API Security Challenges in Mobile Environments

(3/3)

- + Common API Security Issues:
 - + Insufficient authorization checks (BOLA/IDOR)
 - + Excessive data exposure in responses
 - + Insecure direct object references
 - + Token leakage and mismanagement

What Is Client-Server Architecture?

- + Core Components:
 - + Client: Mobile application (Android/iOS) - the presentation layer
 - + Server: Backend systems - application logic and data storage
 - + APIs: The communication bridge between clients and servers

The API Server as the Middle Layer

- + Acts as a gateway between client and backend services
- + Handles:
 - + Request routing
 - + Authentication & authorization
 - + Rate limiting and validation
- + Returns sanitized and structured data (usually JSON)

REST APIs - The Industry Standard

(1/4)

- + Core Principles:
 - + Stateless client-server architecture
 - + Resource-based URLs (e.g. `/user/123`, `/orders/active`)
 - + Standard HTTP methods (`GET`, `POST`, `PUT`, `DELETE`)
 - + Representation formats (typically JSON)

REST APIs - The Industry Standard

(2/4)

- + Strengths for Mobile:
 - + Widespread adoption and developer familiarity
 - + Excellent tooling and library support
 - + HTTP caching mechanisms
 - + Straightforward implementation
 - + Scalability through statelessness

REST APIs - The Industry Standard

(3/4)

- + Limitations in Mobile Contexts:
 - + Multiple round trips for complex data needs
 - + Over-fetching and under-fetching of data
 - + Versioning challenges
 - + Bandwidth inefficiency with verbose JSON

REST APIs - The Industry Standard

(4/4)

- + Security Considerations:
 - + OAuth 2.0 and JWT integration
 - + API key management
 - + Rate limiting implementation
 - + Resource-level authorization

GraphQL - Client-Driven Data Fetching (1/2)

- + Allows clients to query exactly what they need
- + Single endpoint: `/graphql`
- + Uses POST requests with query payloads

GraphQL - Client-Driven Data Fetching (2/2)

- + Pros:
 - + Reduces over-fetching/under-fetching
 - + Great for mobile where bandwidth matters
- + Cons:
 - + Complex introspection surface
 - + Easy to expose hidden fields if access control is weak

What Are Mobile Backend Platforms?

- + Cloud-based services for mobile apps
- + Provide backend functionality without managing infrastructure
- + Common services:
 - + API routing
 - + Authentication
 - + Realtime databases
 - + Push notifications



From a Security Assessment Perspective

- + Firebase: Simple but potentially limited granular control
- + Parse: Flexible but requires proper implementation
- + AWS API Gateway: Enterprise-grade but complex configuration

Mobile Penetration Testing Focus Areas

- + API key storage and transmission
- + Authentication token handling
- + Data access controls and authorization
- + Rate limiting and abuse protection
- + Transport layer security implementation

Introduction to API Surface Mapping

- + The process of identifying all available API endpoints and functionality
- + Crucial for understanding the attack surface
- + Two main approaches:
 - + Static Analysis
 - + Dynamic Analysis

Static Analysis Techniques

- + Involves analyzing app binaries without executing them
- + Use jadx, apktool, class-dump, MobSF
- + Search for:
 - + API base URLs
 - + Endpoints in hardcoded strings
 - + Third-party SDKs and API keys
- + Remember: Obfuscation may limit effectiveness

Dynamic Analysis for API Mapping

- + Observing actual runtime traffic and behavior
- + Methods:
 - + Monitor API calls during app use
 - + Hook networking libraries to extract endpoints
 - + Analyze WebView traffic and in-app browsers
- + Remember: Can miss unused or conditional logic

API Mapping: Banking App Case

- + Static Analysis revealed:
 - + `api.bank.com/v1/internal/debug`
- + Dynamic Analysis revealed:
 - + `/api/transfer/funds`
 - + Unauthenticated access to `/api/user/export`

Creating a Complete API Inventory

- + Methodology:
 - + Begin with static analysis to identify known endpoints
 - + Perform dynamic traffic analysis to confirm and expand findings
 - + Conduct targeted enumeration based on discovered patterns
 - + Document and classify endpoints for security testing
 - + Validate coverage through automated scanning

Comparative Strengths

	Static Analysis	Dynamic Analysis
Hidden Endpoints	Strong for hardcoded URLs	Strong for runtime-generated URLs
Authentication	May reveal mechanisms	Shows actual implementation
Parameters	Identifies expected parameters	Shows actual used parameters
Coverage	Limited by code analysis	Limited by execution paths
Effort Required	Higher initial, lower ongoing	Lower initial, requires active testing

Documentation Best Practices

- + Structured API inventory with endpoint categorization
- + Parameter documentation including types and constraints
- + Authentication requirements for each endpoint
- + Data flow mapping between related endpoints
- + Risk classification based on function and data sensitivity

Understanding API Parameters in Mobile Environments

- + Query Parameters: Appended to URLs (`?key=value&key2=value2`)
- + Path Parameters: Embedded in URL paths (`/users/{id}/profile`)
- + Header Parameters: Custom HTTP headers (`X-API-Key`, etc.)
- + Body Parameters: JSON/XML data structures in request body

```
{ "email": "test@domain.com" }
```

Common Parameter-Based Vulnerabilities

- + Injection Attacks: SQL, NoSQL, LDAP, OS command injection
- + Parameter Tampering: Modifying values to access unauthorized data
- + Mass Assignment: Overposting to modify unintended attributes
- + Type Confusion: Exploiting type casting/conversion issues
- + Parameter Pollution: Multiple instances of the same parameter

Mobile-Specific Parameter Risks

- + Client-side parameter validation bypasses
- + Reverse engineering exposing parameter patterns
- + Plaintext parameters in device logs or caches
- + Insecure interprocess communication exposing API parameters
- + Weaknesses in mobile SDK parameter handling

Fuzzing Methodologies

- + Parameter Type Fuzzing: Sending unexpected data types
- + Boundary Testing: Testing limits (min/max values, buffer sizes)
- + Special Character Injection: Unicode, control characters, SQL wildcards
- + Context-Aware Fuzzing: Platform-specific test cases

Fuzzing Methodologies

+ GET /api/order?id=1005

```
POST /api/transfer
{
  "from_account": "1001",
  "to_account": "1002",
  "amount": "10000"
}
```

Understanding Authorization Vulnerabilities in Mobile APIs (1/2)

- + **IDOR (Insecure Direct Object References)**: Accessing resources via manipulated identifiers
- + **BOLA (Broken Object Level Authorization)**: API-specific term for IDOR in OWASP API Top 10
- + **BFLA (Broken Function Level Authorization)**: Accessing unauthorized functions/endpoints

Understanding Authorization Vulnerabilities in Mobile APIs (2/2)

- + **Horizontal Privilege Escalation:** Accessing other users' data at same permission level
- + **Vertical Privilege Escalation:** Accessing higher-privilege functionality

Understanding the Risks

- + IDOR (Insecure Direct Object Reference)
 - + Accessing someone else's data by modifying object IDs (e.g., /user/123)
- + Enumeration
 - + Predicting and iterating over IDs to find valid ones
- + Privilege Escalation
 - + Gaining higher access by manipulating roles or functions

IDOR/BOLA Testing Methodologies (1/2)

- + Identification Techniques:
 - + Parameter Mapping: Identifying all resource identifiers (IDs, UUIDs, slugs)
 - + Request Tracking: Cataloging endpoint-parameter-object relationships
 - + Response Analysis: Examining data structures for embedded resource identifiers
 - + Access Pattern Analysis: Documenting normal object access workflows

IDOR/BOLA Testing Methodologies

(2/2)

- + Testing Approaches:
 - + ID Substitution: Replace legitimate IDs with other user's resources
 - + CRUD Matrix Testing: Test all operations against various objects
 - + Create, Read, Update, Delete permissions for each resource type
 - + HTTP Method Manipulation: Try alternative HTTP methods on same endpoints
 - + Cross-User Testing: Use multiple accounts to verify access boundaries

Enumeration Techniques

- + `/profile?id=1001`
 - + `for id in {1000..1050}; do curl /api/user/$id; done`
- + Check for:
 - + Differences in status codes
 - + Timing differences
 - + Leaky metadata (usernames, emails)

Testing for Privilege Escalation

- + Horizontal Escalation
 - + Accessing another user's data (IDOR)
- + Vertical Escalation
 - + Performing admin-only actions as a regular user
- + Try accessing `/admin`, `/manage`, `/config` endpoints
- + Add or modify headers/body fields:
 - + `"role": "admin"`
 - + `"is_admin": true`

Evading Detection Controls

- + Rate Limiting Bypass: Distributed requests, timing techniques
- + WAF Evasion: Parameter encoding, request fragmentation
- + Anti-Automation Bypass: Cookie/header manipulation, device fingerprinting
- + IP Restriction Bypass: Proxy rotation, cloud IP usage

Chained Exploits = Greater Impact

- + IDOR + XSS for session hijacking
- + Authentication bypass + privilege escalation
- + Information disclosure + enumeration + IDOR

Documentation & Reporting Best Practices

- + Clear impact demonstration with proof-of-concept
- + Attack chain documentation showing realistic exploitation
- + Quantification of potentially exposed records
- + Remediation guidance for different API architectures

Bug type : UI Redressing

Impact : Phishing

Description :

Clickjacking, also known as a "UI redress attack", is when an attacker uses multiple transparent or opaque layers to trick a user into clicking on a button or link on another page when they were intending to click on the the top level page. Thus, the attacker is "hijacking" clicks meant for their page and routing them to another page, most likely owned by another application, domain, or both.

Using a similar technique, keystrokes can also be hijacked. With a carefully crafted combination of stylesheets, iframes, and text boxes, a user can be led to believe they are typing in the password to their email or bank account, but are instead typing into an invisible frame controlled by the attacker.

```
<html>
<head><title>Clickjack test page</title></head>
<body>

<h1> Clickjacking in your website </h1>
<iframe width="1000" height="500" src="https://www.sparta.com.tr/">
</body>

</html>
```

Impact:

The site can also be opened in an iframe after the user has logged it making it hard for the user to avoid phishing.A user can be tricked into downloading amalicious file that an attacker wants a user to download, allowing an attacker to gain access to the users device .

Remediation :

Add an iframe destroyer in the header of the page

Note :

I am also attaching a screen shot as proof of concept.

I hope to receive reward for the responsible disclosure of the vulnerability

waiting for your response

Best Regards

White HaT

Hello,

Thank you for your concern.

However, as you can see this vulnerability has no impact on our systems or users thus it doesn't qualify for any recognition and/or reward.

Best regards,

Alper

Hi there,

Of Course but this can be used for click baiting you should fix it, and this was kind of an introduction, Can I send further vulnerability reports?

Waiting for your response

Risk Indicators

- + Numeric, sequential, or predictable IDs
- + Lack of resource ownership validation in responses
- + Error messages revealing resource existence
- + Inconsistent authorization across API versions

Response Analysis

- + Status code anomalies indicating parameter issues
- + Error message analysis for information disclosure
- + Performance variation suggesting server-side impact
- + Unexpected data in responses revealing parameter handling flaws

Introduction to Token-Based Authentication

- + Mobile apps use tokens to authenticate API calls
- + Common token types:
 - + Session IDs
 - + JWT (JSON Web Tokens)
 - + OAuth Access & Refresh Tokens
- + Tokens are sent in headers, cookies, or body

Criticality in Mobile Context

- + Longer-lived sessions than web applications
- + Background processing requiring persistent authentication
- + Multiple devices sharing authentication contexts
- + Offline capabilities requiring token caching

Understanding JWT Security Fundamentals

- + JWT Components:
 - + Header: Algorithm and token type
 - + Payload: Claims and data
 - + Signature: Cryptographic validation

Common JWT Vulnerabilities

- + Algorithm Confusion/None Algorithm: Bypassing signature verification
- + Weak Signing Keys: Brute-forcing HMAC secrets
- + Missing Signature Validation: Accepting unsigned tokens
- + Information Disclosure: Sensitive data in payload
- + Missing Claim Validation: Accepting expired/invalid tokens

Mobile-Specific JWT Issues

- + Hardcoded verification keys in app binaries
- + Client-side JWT validation (dangerous)
- + Improper JWT storage in insecure locations
- + Long expiration times for offline functionality

Session Fixation in Mobile Environments

- + Attacker establishes a known session ID
- + Victim is tricked into using the attacker-supplied session
- + Victim authenticates with the fixed session
- + Attacker gains access to authenticated session

Mobile Attack Vectors

- + Deep links with session parameters:

`myapp://login?session=KNOWN_ID`

- + Malicious QR codes: Encoding fixed session identifiers
- + Cross-app request forgery: Forcing parameters via app communication
- + Man-in-the-Middle: Injecting known session values
- + SMS/Email links: Phishing with manipulated parameters



Testing Security Features and Protections

Understanding the Biometric Authentication Flow

- + iOS: Uses the LocalAuthentication framework which communicates with the Secure Enclave
- + Android: Uses BiometricPrompt API (modern) or FingerprintManager (legacy)
- + The OS handles the actual biometric validation, not the app
- + Apps receive only a success/failure response, never the actual biometric data

Verifying biometric implementations (Touch ID / Face ID)

- + Start by understanding how the app uses biometrics:
 - + Does it unlock the app?
 - + Does it authorize specific actions (e.g., bank transfer)?
 - + Is there a fallback (e.g., PIN)?
 - + Is biometric auth tied to a secure session or used just as a shortcut?

Analyze the Implementation (Static Analysis)

(1/4)

- + On Android:
 - + Decompile the APK using JADX or MobSF.
 - + Look for biometric-related classes:
 - + `androidx.biometric.BiometricPrompt`
 - + `FingerprintManager`, `KeyguardManager`

Analyze the Implementation (Static Analysis)

(2/4)

- + Examine what happens after successful biometric auth:
 - + Is a token retrieved?
 - + Is a new session started?
 - + Are any security checks enforced?

Analyze the Implementation (Static Analysis)

(3/4)

- + On iOS:
- + Analyze class names and symbols using class-dump, Frida, or MobSF.
- + Look for:
 - + `LAContext` (used for biometrics)
 - + Methods like `evaluatePolicy:localizedReason:reply:`

Analyze the Implementation (Static Analysis)

(4/4)

- + Look for:
 - + Biometric success leads directly to sensitive operations with no backend validation.
 - + No use of device-bound secure storage (Keychain/Keystore).

Attempt to Bypass Biometric Check

- + Hook biometric callback to always return true
- + Replay or tamper with session tokens captured after biometric auth
- + Tamper fallback authentication (e.g., weak PIN stored insecurely)
- + Disable biometric auth via app settings and attempt to reuse session

Inspect Secure Storage

- + Check if biometric credentials or session tokens are stored insecurely:
 - + Android: Look in SharedPreferences, internal storage, and Keystore
 - + iOS: Dump and inspect Keychain contents (e.g., using keychain_dumper, Frida, or objection)

Checklist for Secure Biometric Implementation

- + Flow Control: Is biometric success validated server-side?
- + Bypass Attempt: Can you force success via hooking?
- + Storage Review: Are tokens or fallback PINs stored securely?
- + Session Integrity: Does biometric auth lead to a valid session?
- + Fallback Method: Is fallback weak or easily guessable?

Brute-force Protections and Rate-limiting

- + Password complexity vs. length
- + Common patterns in mobile login flows:
 - + Username → password
 - + Email/SMS OTP flows
 - + Biometric with fallback

Brute-force Attacks

- + Online vs. offline brute-force
- + Credential stuffing (using leaked password dumps)
- + Password spraying (targeting multiple accounts)
- + Enumeration through timing/detailed error messages

Brute Force Testing

- + Map login flow (observe API calls)
- + Identify parameters for:
 - + Username/email
 - + Password/OTP
 - + Session/token
- + Check if brute forcing is client-side or server-side

Brute Force Testing

- + Unlimited login attempts
- + No lockout mechanism
- + Missing device fingerprinting
- + Session reuse
- + Lack of CAPTCHA

Device Binding and Dynamic Checks

- + Applications often attempt to verify who the user is — and where they are logging in from — by identifying the device

What is Device Binding?

- + Associating a device with a user account or session
- + Helps prevent unauthorized logins or replay attacks
 - + Often used in: Mobile banking apps, authentication flows (e.g., OTP delivery), password recovery restrictions
- + Can be persistent (linked to user ID) or temporary (session-specific)

Dynamic Runtime Checks

- + Apps may perform real-time assessments of the environment during execution
- + Common checks:
 - + Debugger detection
 - + Emulator detection
 - + Frida / hooking frameworks
 - + Root/jailbreak checks
 - + Magisk / Xposed detection

Testing Methodologies: Static Analysis

- + Use tools like JADX, MobSF, or class-dump to analyze:
 - + Where and how device IDs are read (e.g., Settings.Secure.ANDROID_ID)
 - + Custom logic for storing UUIDs
 - + Emulator-specific code blocks (Build.FINGERPRINT, ro.hardware)

Testing Methodologies: Dynamic Analysis

- + Hook device fingerprinting logic using Frida:

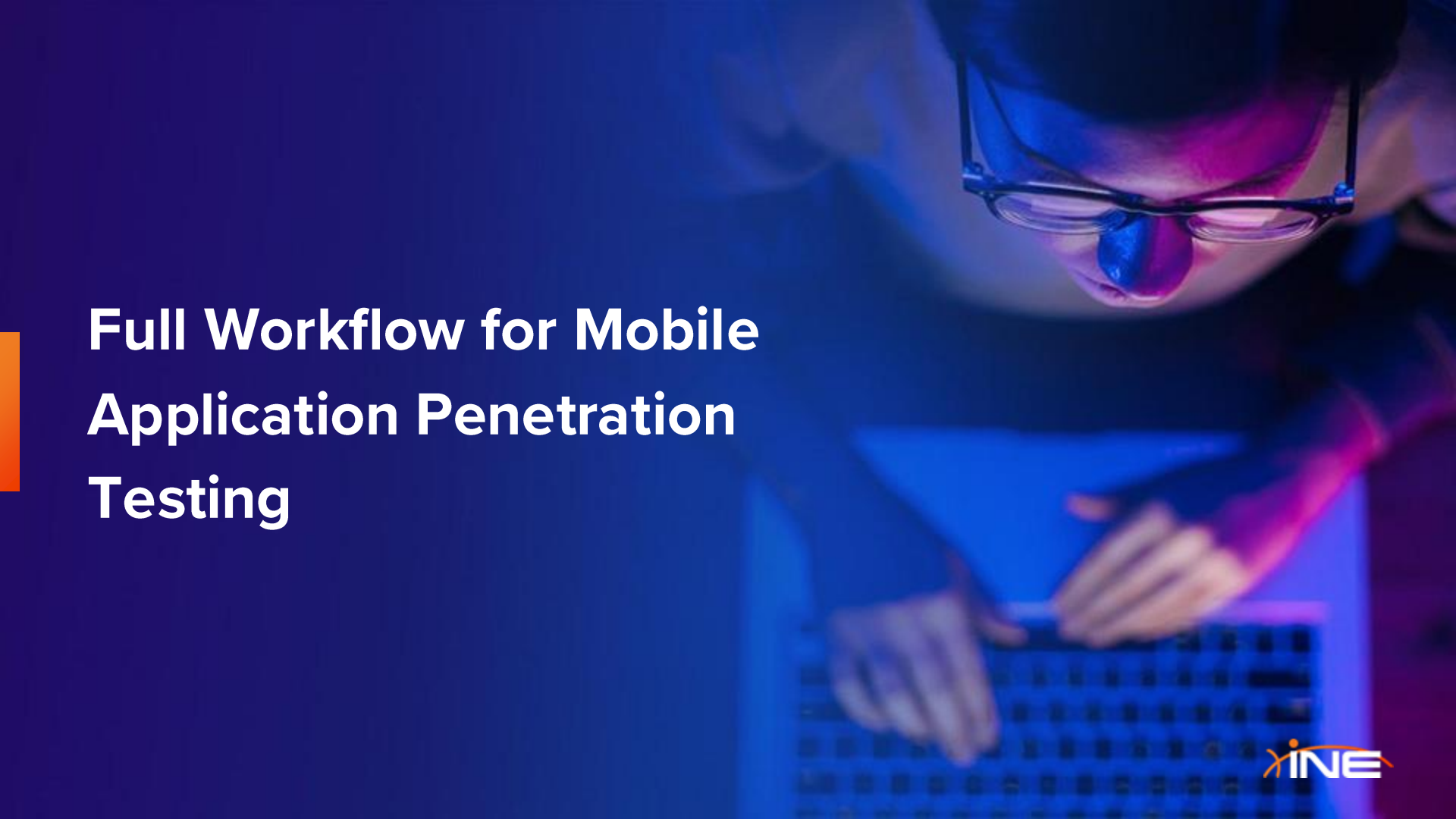
```
Java.perform(function () {  
    var secureSettings = Java.use("android.provider.Settings$Secure");  
    secureSettings.getString.overload('android.content.ContentResolver',  
    'java.lang.String').implementation = function (cr, name) {  
        if (name == "android_id") {  
            console.log("Bypassing android_id fingerprinting");  
            return "fake-device-id-1234";  
        }  
        return this.getString(cr, name);  
    };  
});
```

Testing Methodologies: Multiple Device Simulation

- + Run multiple Android emulators with different IDs (AVD, Genymotion, etc.)
- + Simulate user logging in from different “devices”
- + Observe server behavior:
 - + Is OTP blocked on second emulator?
 - + Does the app log out or throw an error?

Why Root/Jailbreak Detection Exists?

- + Protect sensitive data and secure storage (Keychain, Keystore)
- + Prevent code tampering, instrumentation, or binary patching
- + Protect DRM, anti-cheat mechanisms, and IP in gaming/streaming apps
- + Prevent bypassing business logic (e.g., free purchases, license skipping)
- + Meet compliance or security requirements (e.g., PCI DSS, banking apps)



Full Workflow for Mobile Application Penetration Testing

Scoping, Planning, and Preparing Mobile Pentest Engagements

- + Initial Client Communication
 - + Identifying key stakeholders and technical contacts
 - + Conducting preliminary scoping calls
 - + Determining testing objectives and business priorities
 - + Understanding regulatory requirements (GDPR, HIPAA, PCI DSS, etc.)
 - + Evaluating client's security maturity level

Scoping, Planning, and Preparing Mobile Pentest Engagements

- + Application Profiling
 - + Identifying application type (native, hybrid, web-wrapper)
 - + Determining target platforms (iOS, Android, multiple)
 - + Cataloging application features and functionality
 - + Identifying backend services and integrations
 - + Documenting authentication mechanisms and data sensitivity

Scoping, Planning, and Preparing Mobile Pentest Engagements

- + Comprehensive Scoping Questionnaire
 - + Application architecture and technologies
 - + Available testing environments (prod, staging, dev)
 - + Account provisioning requirements
 - + API documentation and specifications
 - + Source code availability
 - + Previous assessment history and known issues
 - + Special handling requirements (PHI, PII, financial data)

Scoping, Planning, and Preparing Mobile Pentest Engagements

- + Defining Testing Boundaries
 - + In-scope components (mobile client, APIs, backends)
 - + Out-of-scope elements (third-party services, cloud infrastructure)
 - + Permitted testing activities
 - + Restricted or prohibited testing techniques
 - + Sensitive functionality requiring special handling

Engagement Planning and Resource Allocation

- + Effort Estimation
 - + Complexity-based estimation models
 - + Feature-based estimation approaches
 - + Historical comparison with similar applications
 - + Risk-weighted testing allocation
 - + Time allocation for different testing phases

Engagement Planning and Resource Allocation

- + Testing Timeline Development
 - + Establishing key milestones and deliverables
 - + Creating a workable testing schedule
 - + Allocating buffer time for unexpected complexities
 - + Coordinating with client release cycles
 - + Planning status update touchpoints

Engagement Planning and Resource Allocation

- + Resource Requirements
 - + Hardware requirements (test devices, specific models)
 - + Software and tool requirements
 - + Environment access needs
 - + Test account provisioning
 - + Team composition based on application complexity

Engagement Planning and Resource Allocation

- + Rules of Engagement Documentation
 - + Testing timeline and schedule
 - + Communication protocols during testing
 - + Escalation procedures for critical findings
 - + Emergency contact information
 - + Status reporting frequency and format
 - + Issue disclosure and handling process

Testing Environment Preparation

- + Mobile Device Lab Setup
 - + Physical device vs. emulator considerations
 - + Required OS versions and variants
 - + Device management and preparation
 - + Rooted/jailbroken device preparation
 - + Baseline configuration and reset procedures

Testing Environment Preparation

- + Testing Toolset Preparation
 - + Proxy setup (Burp Suite, OWASP ZAP, Charles)
 - + Static analysis tools configuration (MobSF, Insider)
 - + Dynamic analysis framework preparation (Frida, Objection)
 - + Traffic analysis tools (Wireshark, tcpdump)
 - + Custom script development for test-specific needs

Testing Environment Preparation

- + Network Infrastructure Setup
 - + Isolated testing network configuration
 - + Certificate installation for TLS interception
 - + VPN configuration (if required)
 - + Host machine preparation
 - + Baseline traffic capture for comparison

Testing Environment Preparation

- + Documentation and Evidence Collection Systems
 - + Setting up secure evidence storage
 - + Establishing finding documentation templates
 - + Implementing screenshot and screen recording procedures
 - + Configuring log capture mechanisms
 - + Preparing collaborative workspaces for team testing

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Public Information Analysis
 - + App store metadata examination
 - + Developer website and documentation review
 - + Job posting analysis for technology insights
 - + Social media and public repository analysis
 - + Previous vulnerability disclosures review

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Application Binary Acquisition
 - + App store download procedures
 - + Alternative markets and historical version retrieval
 - + Direct APK/IPA acquisition from client
 - + Handling enterprise distribution applications
 - + TestFlight/Beta application access

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Initial Documentation Review
 - + API documentation analysis
 - + SDK implementation guidelines review
 - + Release notes and change log examination
 - + Known issues and limitations review
 - + Third-party component identification

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Application Behavior Baseline
 - + Normal user workflow documentation
 - + Functional testing to understand application flow
 - + Performance and responsiveness baseline
 - + Network traffic patterns during normal operation
 - + Authentication and session management observation

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Application Decompile and Analysis
 - + Android APK unpacking and decompilation
 - + iOS IPA extraction and analysis
 - + Manifest and info.plist examination
 - + Permissions and entitlements review
 - + Component identification and architecture mapping

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Static Code Analysis
 - + Hardcoded sensitive information discovery
 - + Security control identification
 - + Authentication mechanism analysis
 - + Encryption implementation review
 - + API endpoint enumeration

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Network Communications Mapping
 - + Initial traffic capture and analysis
 - + API endpoint cataloging
 - + Authentication flow documentation
 - + Data transmission patterns
 - + Certificate pinning detection

Discovery, Information Gathering, and Pre-Exploitation Setup

- + Backend System Identification
 - + Server technology fingerprinting
 - + Cloud service identification
 - + Third-party API integration discovery
 - + Authentication server identification
 - + Database and storage system detection

Attack Surface Mapping

- + Client-Side Attack Surface
 - + Input validation points identification
 - + Local storage mechanisms
 - + Inter-app communication channels
 - + WebView implementations
 - + Custom URL scheme handlers

Attack Surface Mapping

- + Network Attack Surface
 - + API endpoint inventory
 - + Authentication mechanisms
 - + Session management approaches
 - + Data transmission protocols
 - + Certificate validation implementations

Attack Surface Mapping

- + Backend Attack Surface
 - + Server-side API implementation
 - + Authentication and authorization systems
 - + Data storage mechanisms
 - + Business logic implementation
 - + Integration points with other systems

Attack Surface Mapping

- + Threat Modeling
 - + Creating data flow diagrams
 - + Identifying trust boundaries
 - + STRIDE-based threat identification
 - + Risk prioritization for testing
 - + Attack tree development for complex scenarios

Risk-Based Test Planning

- + Risk Assessment and Prioritization
 - + Asset identification and valuation
 - + Threat scenario development
 - + Vulnerability likelihood estimation
 - + Impact assessment for different attack vectors
 - + Risk scoring and prioritization

Risk-Based Test Planning

- + Criticality-Based Test Case Prioritization
 - + Identifying high-risk functionality
 - + Authentication and authorization focus areas
 - + Sensitive data handling test prioritization
 - + Client-specified concerns and priorities
 - + Balancing breadth vs. depth in testing

Risk-Based Test Planning

- + Attack Scenario Development
 - + Creating realistic attack narratives
 - + Chaining potential vulnerabilities
 - + Considering attacker motivation and capabilities
 - + Developing proof-of-concept attack plans
 - + Impact demonstration planning

Walkthrough of Full Pentest with Reporting Checkpoints

1. Engagement Kickoff and Initial Setup
2. Core Testing Execution
3. Advanced Testing and Exploitation
4. Specialized Testing Areas
5. Report Preparation and Delivery

1 - Engagement Kickoff and Initial Setup

- + Client Kickoff Meeting
 - + Introduction of testing team
 - + Review of scope and objectives
 - + Confirmation of timeline and deliverables
 - + Initial credential and access provision
 - + Communication channels establishment

1 - Engagement Kickoff and Initial Setup

- + Initial Reconnaissance Execution
 - + Application installation and configuration
 - + Initial application usage and mapping
 - + Preliminary traffic capture and analysis
 - + Application decomposition and static analysis
 - + Environment and tool setup verification

1 - Engagement Kickoff and Initial Setup

- + First Checkpoint: Scope Validation
 - + Confirming test environment readiness
 - + Validating application version and build
 - + Ensuring access to required systems
 - + Identifying any scope adjustments needed
 - + Preliminary findings and approach adjustment

1 - Engagement Kickoff and Initial Setup

- + Test Plan Refinement
 - + Adjusting test plan based on initial reconnaissance
 - + Refining risk assessment based on observed implementation
 - + Updating test case priorities
 - + Addressing any newly discovered components
 - + Finalizing testing timeline based on complexity assessment

2 - Core Testing Execution

- + Authentication and Authorization Testing
 - + Authentication mechanism analysis
 - + Session management testing
 - + Authorization control validation
 - + Role-based access testing
 - + Multi-factor authentication assessment

2 - Core Testing Execution

- + Data Storage and Protection Testing
 - + Local data storage security assessment
 - + Encryption implementation validation
 - + Sensitive data handling verification
 - + Backup data security testing
 - + Data leakage testing

2 - Core Testing Execution

- + Network Communication Testing
 - + Transport security verification
 - + API endpoint security testing
 - + SSL/TLS implementation validation
 - + Certificate pinning assessment
 - + Backend communication security

2 - Core Testing Execution

- + Second Checkpoint: Critical Finding Review
 - + Early critical finding documentation
 - + Preliminary risk assessment validation
 - + Client notification of significant issues
 - + Adjustment of testing focus based on findings
 - + Progress assessment against test plan

3 - Advanced Testing and Exploitation

- + Platform Interaction Security
 - + Inter-process communication testing
 - + Custom URL scheme handler testing
 - + Clipboard security assessment
 - + Intent/deeplink vulnerability testing
 - + WebView implementation security

3 - Advanced Testing and Exploitation

- + Business Logic Testing
 - + Workflow bypass attempts
 - + Business rule validation testing
 - + Transaction security testing
 - + Race condition testing
 - + Logic flaw hunting and exploitation

3 - Advanced Testing and Exploitation

- + Client-Side Injection Testing
 - + WebView-based injection testing
 - + Local storage injection
 - + XML/JSON parser manipulation
 - + Custom protocol handler injection
 - + Input validation bypass attempts

3 - Advanced Testing and Exploitation

- + Third Checkpoint: Exploitation Progress
 - + Successful exploitation documentation
 - + Attack chaining assessment
 - + Exploitation evidence collection
 - + Impact demonstration preparation
 - + Test coverage validation

4 - Specialized Testing Areas

- + Binary Protection Assessment
 - + Code obfuscation effectiveness testing
 - + Anti-tampering control assessment
 - + Reverse engineering resistance testing
 - + Root/jailbreak detection bypass
 - + Runtime manipulation prevention testing

4 - Specialized Testing Areas

- + Privacy Testing
 - + Data collection assessment
 - + Permission usage validation
 - + Tracking mechanism identification
 - + Privacy policy compliance verification
 - + Unnecessary data exposure testing

4 - Specialized Testing Areas

- + Backend API Security Testing
 - + API security control assessment
 - + Injection vulnerability testing
 - + Authentication and authorization testing
 - + Rate limiting and resource control testing
 - + Business logic flaw identification

4 - Specialized Testing Areas

- + Fourth Checkpoint: Comprehensive Testing Status
 - + Testing coverage assessment
 - + Finding categorization and prioritization
 - + Additional testing needs identification
 - + Client update on overall security posture
 - + Remediation planning discussion initiation

5 - Report Preparation and Delivery

- + Finding Validation and Categorization
 - + Vulnerability reproduction and validation
 - + False positive elimination
 - + Risk rating assignment
 - + Finding categorization by type and component
 - + Exploitation proof refinement

5 - Report Preparation and Delivery

- + Technical Report Development
 - + Detailed vulnerability documentation
 - + Reproduction steps clarification
 - + Technical evidence inclusion
 - + Root cause analysis
 - + Detailed remediation recommendations

5 - Report Preparation and Delivery

- + Executive Summary Creation
 - + Overall risk assessment
 - + Key finding highlighting
 - + Strategic remediation roadmap
 - + Comparative security posture assessment
 - + Business impact explanation

5 - Report Preparation and Delivery

- + Final Checkpoint: Report Delivery and Review
 - + Report walkthrough with technical team
 - + Finding explanation and clarification
 - + Remediation strategy discussion
 - + Retest planning and scheduling
 - + Engagement closeout and feedback collection

Mobile Application Penetration Testing Techniques

Key Concepts

- + Penetration testing methodologies for mobile apps
- + Dynamic testing for Android and iOS
- + Real-world attack simulation and MASVS alignment
- + API and backend testing for mobile applications
- + Reporting and remediation strategies



LEARNING OUTCOMES

- + Identify and exploit vulnerabilities in mobile apps using real tools
- + Map findings to MASVS and industry standards
- + Execute advanced dynamic testing on Android and iOS
- + Conduct and document complete mobile penetration tests

Real-World Applications

- + Pen-testing enterprise mobile applications
- + Assessing security of consumer-facing mobile APIs
- + Preparing vulnerability assessments and remediation reports for clients

Next Steps

- + Advance to mobile red teaming or threat modeling courses
- + Practice in mobile CTF or bug bounty environments
- + Deepen skills with secure mobile development courses

THANKS FOR WATCHING!



EXPERTS AT MAKING YOU AN EXPERT

